

University of Wollongong
**School of Electrical, Computer and
Telecommunications Engineering**

ECTE351 Deliverable Cover Sheet

Team Name: Team 11

Subject Number and Name: ECTE351 - Engineering Design and Management
3

DELIVERABLE NUMBER AND TOPIC Assignment 10 - Final Project
Report - Micromax: Emergency Response Systems for Organisations

Date Lodged: 24/10/25

Team Members (list only group members who have taken part in production of the deliverable):

1. Zachary Wark
2. Thomas Speer
3. Nur Azmina Mohd Rashidi
4. Oliver Michels
5. Ziyue Li
6. Laila Salem

Team Member Submitting: Name: Zachary Wark

ID No. 6784471

Signature:



Grade:

Please Note: Assignments are not to be submitted by fax.

Effective Summary

The Micromax Emergency Response system is a wearable device and central hub being designed to improve workplace safety through live monitoring, automatic alerts and workplace safety tracking. The systems combined biometric and motion sensors linked to cloud-based monitoring to detect emergencies quickly, alert responders and record accurate incident data for workers. This ensures that individuals working in high risk and isolated environments such as construction sites or warehouses, can receive immediate assistance and medical attention when accident or sudden health emergencies occur, significantly reducing response time and improving safety outcomes.

Unlike existing devices that depend on subscription fees, The Micromax Emergency Response system offers a one-time purchase model that provides long term cost efficiency and facilitates adoption among organisations with constrained budgets or fixed capital expenditures cycles. A fully functional prototype was implemented featuring heart rate and motion detection automated emergency alert through eSIM integration, a live HTML based monitoring dashboard for real time data display for supervisors and safety officers. Together, these features show the practicality of linking low power wearables sensors with a centralized data management platform to optimise emergency response time, increase operational visibility and workplace safety compliance.

Although the project faced challenges such as GPS integration, sensor calibration and wireless communication stability, the final prototype demonstrates strong feasibility and readiness for real world application. Future development will prioritise improving sensor reliability, reducing device size, refining user interface functionality to provide a more intuitive and user-friendly experience for monitoring and control. Additionally, future designs will be developed to achieve full compliance with Australia regulatory standards including Therapeutic Goods Administration (TGA) and Regulatory Compliance Mark (RCM) compliance for electrical safety and wireless communications standards. The project highlights strong potential for scalable implementation across various workplaces and represents a practical commercial pathway toward improved Work Health and Safety performance through reduced response times, accurate incident reporting and compliance tracking.

List of Figures

Figure 1: Product Circuit Design.....	8
Figure 2: Raspberry Pi 5 Model B (8GB).....	9
Figure 3: Raspberry Pi 5 Power Supply.....	9
Figure 4: Raspberry Pi Pico 2 W.....	10
Figure 5: Biometric Sensor.....	10
Figure 6: 3-Axis Gyroscope and Accelerometer.....	11
Figure 7: USB-C LiPo Battery Charger.....	11
Figure 8: Lithium Polymer Ion Battery.....	11
Figure 9: OLED Display.....	12
Figure 10: Mini Push Button Switch.....	12
Figure 11: Emergency Response Dashboard.....	13
Figure 12: Developed Work Breakdown Structure.....	18
Figure 13: Return on Investment Per Wearable Device.....	21
Figure 14: Wearable Device Base Layer.....	55
Figure 15: Wearable Device Raspberry Pi Layer.....	55
Figure 16: Wearable Device Charger Layer.....	55
Figure 17: Wearable Device BatteryLayer.....	55
Figure 18: Wearable Device GPS Layer.....	55
Figure 19: Wearable Device OLED Layer.....	55
Figure 20: Wearable Device OLED Cover Top View.....	56
Figure 21: Wearable Device OLED Cover Bottom View.....	56
Figure 22: Wearable Inner Rail Close Up.....	56
Figure 23: Wearable Outer Rail Close Up.....	56
Figure 24: Wearable Device Assembly Top View.....	56
Figure 25: Wearable Device Assembly Bottom View.....	56
Figure 26: Base Layer Top View.....	57
Figure 27: Base Layer Bottom View.....	57
Figure 28: Pico Layer Top View.....	57
Figure 29: Pico Layer Bottom View.....	57
Figure 30: Charger Layer Top View.....	57
Figure 31: Charger Layer Bottom View.....	57
Figure 32: Screen Layer Top View.....	58
Figure 33: Screen Layer Bottom View.....	58
Figure 34: Assembled Device Top View.....	58
Figure 35: Assembled Device Bottom View.....	58
Figure 36: Central Hub Overall Dimension Drawing.....	59
Figure 37: Disassembled Housing With Corresponding Components Beside.....	60
Figure 38: Disassembled Housing With Corresponding Components In Place.....	60
Figure 39: Assembled Housing With Components In Place.....	61
Figure 40: Central Hub Top View.....	61

Figure 41: Central Hub Bottom View.....	61
Figure 42: Central Hub Assembly.....	62
Figure 43: Bottom Layer Top View.....	62
Figure 44: Bottom Layer Bottom View.....	62
Figure 45: Top Layer Top View.....	62
Figure 46: Top Layer Bottom View.....	62
Figure 47: Assembled Device Top View.....	63
Figure 48: Assembled Device Bottom View.....	63
Figure 49: Central Hub Overall Dimension Drawing.....	63
Figure 50: Disassembled Housing.....	64
Figure 51: Assembled Housing.....	64
Figure 52: Order Production Cost.....	65
Figure 53: Relative Order Production Cost Per Wearable Device.....	66
Figure 54: Return on Investment Per Wearable Device.....	66
Figure 55: Gantt Chart.....	67

List of Tables

Table 1: Roles of Each Team Member.....	18
Table 2: Comprehensive Expenditure Record.....	19
Table 3: Wearable Device Final Components List.....	20
Table 4: Central Hub Final Components List.....	20
Table 5: Man Hour Expenditures.....	21

Table of Contents

University of Wollongong.....	1
Effective Summary.....	2
List of Figures.....	3
List of Tables.....	4
Table of Contents.....	5
1.0 Introduction.....	7
2.0 Market Research.....	7
2.1 Marketing Strategy.....	7
2.2 Competitor Analysis.....	8
2.3 Product Evaluation.....	8
3.0 Product Design.....	8
3.1 Overview.....	8
3.2 Hardware.....	9
3.2.1 Central Hub.....	9
3.2.1.1 Raspberry Pi 5 Model B (8GB).....	9
3.2.1.2 Raspberry Pi 5 Power Supply (5.1V USB-C).....	9
3.2.1.3 Housing.....	9
3.2.2 Wearable Device.....	10
3.2.2.1 Raspberry Pi Pico 2 W.....	10
3.2.2.2 Biometric Sensor (MAX30102).....	10
3.2.2.3 3-Axis Gyroscope and Accelerometer (MPU6050).....	11
3.2.2.4 USB-C LiPo Battery Charger (Makerverse).....	11
3.2.2.5 Lithium Polymer Ion Battery (3.7V, 400mAh).....	11
3.2.2.6 OLED Display (SSD1306).....	12
3.2.2.7 Mini Push Button Switch.....	12
3.2.2.8 Housing.....	12
3.2.2.9 Future PCB Implementation.....	12
3.3 Software.....	13
3.3.1 Front End.....	13
3.3.1.1 HTML.....	13
3.3.1.2 Emergency Override Button.....	14
3.3.1.3 Server.....	14
3.3.2 Back End.....	14
3.3.2.1 eSIM/SIM module.....	14
3.3.2.2 Accelerometer/ Gyroscope.....	14
3.3.2.3 Heart Rate Sensor.....	15
3.3.2.4 GPS.....	15
3.3.2.5 Raspberry Pi Pico Code.....	15
3.3.2.6 Raspberry Pi 5 Code.....	15
4.0 Project Results.....	16
5.0 Project Management.....	16
5.1 Project Management Overview.....	16
5.2 Project Management Techniques.....	17

5.2.1 Gantt Chart.....	17
5.2.2 Work Breakdown Structure.....	17
5.2.3 Weekly Meeting Minutes.....	18
6.0 Budget.....	18
6.1 Expenditure Record.....	19
6.1.1 Comprehensive Expenditure Record.....	19
6.1.2 Final Components List.....	20
6.2 Man Hour Expenditures.....	21
6.3 Cash Flow Forecasts.....	21
7.0 Ethical Considerations.....	22
8.0 Sustainability.....	23
8.1 Environmental sustainability.....	23
8.2 Social Sustainability.....	23
8.3 Economic Sustainability.....	23
9.0 Conclusion and Recommendations.....	24
10.0 Individual Contribution.....	25
10.1 Laila Salem (7545447).....	25
10.2 Thomas Speer (7679877).....	25
10.3 Oliver Michels (7962228).....	26
10.4 Zachary Wark (6784471).....	26
10.5 Nur Azmina Mohd Rashidi (7754127).....	27
10.6 Ziyue Li.....	27
11.0 References.....	27
12.0 Appendices.....	28
12.1 Appendix A: Wearable Code (Pi Pico).....	28
12.2 Appendix B: Central Hub Code (Pi 5).....	35
12.3 Appendix C: Wearable Device Housing.....	55
12.3.1 Initial CAD Images.....	55
12.3.2 Final CAD Images.....	57
12.3.3 Final Dimensioned Drawing.....	59
12.3.4 Final Physical Images.....	60
12.4 Appendix D: Central Hub Housing.....	61
12.4.1 Initial CAD Images.....	61
12.4.2 Final CAD Images.....	62
12.4.3 Final Overall Dimension Drawing.....	63
12.4.4 Final Physical Images.....	64
12.5 Appendix E: Cash Flow Forecast Calculations.....	65
12.6 Appendix F: Gantt Chart.....	67
12.7 Appendix G: FCS.....	68

1.0 Introduction

Workplace is a major concern across industries, especially in sectors where employees operate independently or in high-risk condition. Undetected incidents and delayed emergency response can lead to increased injury severity, extended recovery periods and operational disruption. The Micromax Emergency Response System was developed to address these challenges through an integrated wearable and central hub system that provides real time monitoring, automated emergency notification, accurate incident tracking. The system uses biometric sensors, motion detection and wireless communication technologies to detect emergencies such as falls or irregular vital signs. When an event is detected, data is automatically transmitted to the central hub, where it is processed and displayed through an HTML-based dashboard. At same time automated text alerts are sent to supervisors via an eSIM service. This project was undertaken as part of the ECTE351 subject in collaboration with Micromax Technology, to develop and demonstrate practical and scalable engineering solutions aligned with industry safety requirements. This report presents the market research, product design, development, testing, and evaluation of the prototype, along with key project management insights and recommendations for future improvement.

2.0 Market Research

2.1 Marketing Strategy

Our marketing strategy focuses on demonstrating real-world value in high-risk workplaces while lowering adoption friction for decision-makers. We will prioritise industries such as construction and large warehouse manufacturing, while also addressing broader lone-worker scenarios across sectors where employees may be isolated or intermittently separated from coworkers. To convert interest into purchase intent, we will provide free live demonstrations—in settings like the innovation fair and, upon request, on-site trials—so safety managers and executives can see real-time injury detection, automated emergency alerts, GPS tracking, and seamless integration with existing WHS systems working in their own environments. To further de-risk procurement, we will offer a free trial or a money-back guarantee, reinforcing confidence that the device reduces response times and improves incident documentation with accurate timestamps and event traces. In parallel, we will explore collaborations with insurance companies to recognise reduced liability and improved reporting; where viable, insurers may support premium reductions for workplaces adopting our system. Pricing will differentiate us from subscription-based incumbents by positioning a one-time upfront payment for the device and installation, which avoids ongoing fees and addresses a common deterrent to long-term adoption. Throughout, our messaging will emphasise how the system helps employers meet Work Health and Safety obligations by providing a practical, auditable tool that supports compliance and reduces legal risk, while our product roadmap highlights customisation options that let organisations configure features to their operational needs.

2.2 Competitor Analysis

The current competitive set includes Blackline Safety, SoloProtect, and Wearin', whose offerings address aspects of lone-worker protection but differ from our proposed solution in target segments and core feature emphasis. Many existing models are anchored to subscription plans, which can deter long-term adoption due to cumulative costs and budgeting uncertainty. By contrast, our one-time payment model eliminates recurring fees and aligns better with capital expenditure cycles typical in construction and warehousing. Feature-wise, our system combines real-time injury detection, automated alerting to emergency services, GPS tracking, and integration with workplace health and safety systems, while explicitly positioning compliance support under WHS as a primary value driver. This compliance-first framing, together with precise incident documentation, strengthens the business case beyond safety alone by reducing legal exposure and improving audit readiness. Finally, our commitment to deep customisation allows organisations to tune features to their operational constraints, creating a differentiated fit where competitors often provide more fixed bundles.

2.3 Product Evaluation

From a value perspective, the product directly addresses three enterprise priorities: faster emergency response, better incident documentation, and demonstrable WHS compliance. The system's real-time detection and automated alerts can compress the interval between incident and intervention for both employers and emergency services, while GPS and event logging provide accurate, auditable records that support investigations and policy improvement. Customisability ensures the solution scales across industries—from construction sites to nursing homes where individuals face elevated medical risks—allowing the same core platform to be configured for different hazard profiles. On compliance and regulation, the product roadmap acknowledges Australian requirements: if the device is classified as a medical device for fall detection or health monitoring, it must meet TGA regulations and ARTG listing, and all electrical/wireless components must meet RCM standards for safety, EMC, and radio. These guardrails shape our design choices toward robust data handling and system reliability. We also recognise potential drawbacks—upfront cost, privacy concerns, and false alerts—and we plan to mitigate them through a one-time pricing model that avoids recurring liabilities, stronger data-security architecture to address confidentiality, and algorithmic refinement plus user-tunable thresholds to reduce nuisance alarms. Given the market context—an estimated AUD 28.8 billion workplace-safety technology market in 2024 with projected growth to nearly AUD 70 billion by 2030 at a 16.9% CAGR—the product is well-timed to capture demand as standards tighten and organisations invest in scalable safety solutions. Looking forward, the evaluation supports a roadmap that extends beyond wearables to multiple device forms and expands detectable event types, guided by field feedback on what companies value most.

3.0 Product Design

3.1 Overview

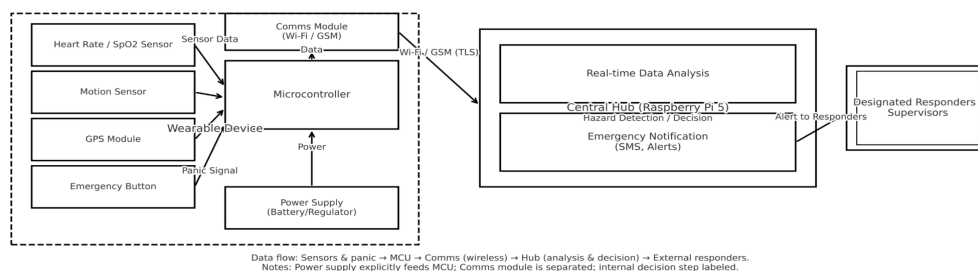


Figure 1: Product Circuit Design

3.2 Hardware

3.2.1 Central Hub

3.2.1.1 *Raspberry Pi 5 Model B (8GB)*

The central hub runs entirely off of a Raspberry Pi 5 Model B (8GB) single board computer (SBC), which includes the practical key features:

- 2.4/5GHz dual-band Wi-Fi 5 (300Mbps)
- Bluetooth 5
- Dedicated Raspberry Pi fan connector
- 2 x USB3.0 ports
- 8GB Ram
- BLE support
- USB-C power delivery
- 2 x USB2.0 ports



Figure 2: Raspberry Pi 5 Model B (8GB)

3.2.1.2 *Raspberry Pi 5 Power Supply (5.1V USB-C)*

As the entirety of the central hub runs off of the Raspberry Pi 5 Model B, it is required that the official Raspberry Pi 5 Power Supply is used, as Raspberry Pi themselves state that the use of third-party power supplies may compromise the performance of the SBC.



Figure 3: Raspberry Pi 5 Power Supply

3.2.1.3 *Housing*

The housing for the central hub (As shown in [Appendix D: Central Hub Housing](#)) is a two part, 3D printed case, connected using snap fit joints. The bottom layer includes a vented base to allow for appropriate airflow, a male locating rail along the inner-perimeter of the top edge, locating pins to hold the SBC in place and off of the base (ensuring even air flow access), and two snap fit arms extending upwards from the sides of the case. The top layer includes a vented roof (featuring the Micromax technology logo) to allow for the appropriate airflow, a female locating rail along the bottom inner-perimeter edge, port openings for the USB-C power delivery, both of the USB3.0 ports, and both of the USB2.0 ports. Additionally the top layer has a groove on either side to allow for the connection of the snap fit arms from the bottom layer. The central hub housing is printed out PLA, which is a bio-degradable plastic derived from renewable sources, to ensure the maintenance of the Engineers Australia Code of Ethics section 4.2: “*Practise engineering to foster the health, safety and wellbeing of the community and the environment*”.

3.2.2 Wearable Device

3.2.2.1 Raspberry Pi Pico 2 W

All components of the wearable device are to run off of the Raspberry Pi Pico 2 W microcontroller, powered by an external lithium polymer ion battery, which includes the practical key features:

- 2.4GHz Wi-Fi 4
- Bluetooth 5.2
- 12 possible I2C channels
- Input voltage range 1.8-5.5V (DC)
- Drag & drop microPython programming using mass storage over USB-C

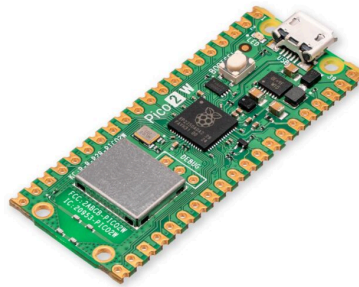


Figure 4: Raspberry Pi Pico 2 W

3.2.2.2 Biometric Sensor (MAX30102)

The biometric sensor, which implements a MAX30102, is housed within the bottom layer of the wearable device housing (with an appropriate opening to allow for readings). It requires an input voltage of 3.3-5V integrating perfectly with the microcontroller's 3V3 voltage supply, and features a 5.6mm x 3.3mm x 1.55mm optical module containing red and infrared LEDs and photodetectors, as well as low noise electronics with ambient light rejection. It runs off of the I2C protocol making use of the microcontroller's internal clock to allow for a waveform pulse measurement, this waveform can then be processed through pulse wave analysis to calculate the users heart rate in BPM.

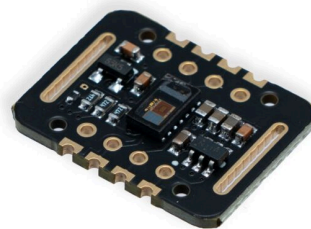


Figure 5: Biometric Sensor

3.2.2.3 3-Axis Gyroscope and Accelerometer (MPU6050)

The MPU6050, is housed within the bottom layer of the wearable device housing alongside the MAX30102. It includes an onboard power regulator and requires an input voltage of 3-5V, integrating perfectly with the microcontroller's 3V3 voltage supply, runs off of the I2C protocol making use of the microcontroller's internal clock to calculate the configurable gyroscopic output range of ± 250 , 500, 1000, or 2000 degrees per second, as well as the configurable acceleration output range of ± 2 , 4, 6, or 8 G's.



Figure 6: 3-Axis Gyroscope and Accelerometer

3.2.2.4 USB-C LiPo Battery Charger (Makerverse)

The Makerverse charger has a selectable range of charging currents of 50mA, 150mA, 400mA, or 500mA, with a regulator providing a constant 3.3V external output from the connected battery, in addition to a 5V USB bus output to the battery when connected to an external power supply via the USB-C cable, perfectly integrating with the microcontrollers 3V3 power supply.

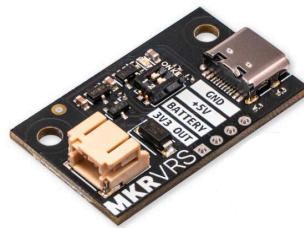


Figure 7: USB-C LiPo Battery Charger

3.2.2.5 Lithium Polymer Ion Battery (3.7V, 400mAh)

The lithium polymer ion battery has cell voltage of 3.7V and a 400mAh capacity, aligning perfectly with the 1.8-5.5V input range for the Raspberry Pi Pico 2 W. Pairing this with the Raspberry Pi Pico 2 W's typical current draw of approximately 10mA when idle, and 35-55mA while running, with a peak of approximately 70mA during active wireless use, the 400mAh capacity of the lithium polymer ion battery allows for approximately 5.7 hours of use on a single charge for a constant current draw of 70mA.

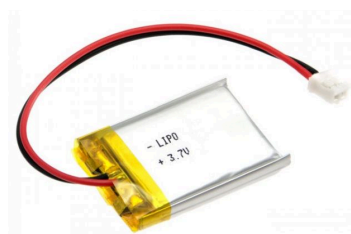


Figure 8: Lithium Polymer Ion Battery

3.2.2.6 OLED Display (SSD1306)

The OLED display module features a 0.96 inch, 64x128 pixel screen with a required input voltage of 3.3V, allowing for appropriate integration with the microcontroller's 3V3 voltage supply, in addition to running off of the I2C protocol ensuring compatibility with the microcontroller.

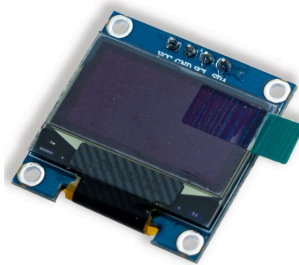


Figure 9: OLED Display

3.2.2.7 Mini Push Button Switch

The small size of the mini push button switch aligns perfectly with the size requirements of the device housing. Additionally, the button being rated for up to 50mA ensures the suitability for its integration with the microcontroller within the device.



Figure 10: Mini Push Button Switch

3.2.2.8 Housing

The housing for the wearable device (as shown in [Appendix C: Wearable Device Housing](#)) is a modular, partially hollow, 3D printed design, comprising multiple layers, each of which are specifically designed for their respective components housed within them, containing locating pins and port holes respective to their components requirements. These layers make use of locating rails, as well as M3 threaded holes to allow for simple assembly/disassembly, ensuring a future proof design with easy access to internal components for upgrade and/or replacement where necessary. Similarly to the central hub housing, the wearable device housing is to be printed out of PLA, which is a bio-degradable plastic derived from renewable sources, to ensure the maintenance of the Engineers Australia Code of Ethics section 4.2: “*Practise engineering to foster the health, safety and wellbeing of the community and the environment*”.

3.2.2.9 Future PCB Implementation

Despite the modular design of the wearable device housing allowing for ease of upgrade and/or replacement of the components, it does simultaneously result in a large requirement for the dimensions for the wearable device housing. Beyond the proof of concept of the current design, further innovation including the implementation of a singular PCB to accommodate for all components within the wearable device (disregarding the battery and microcontroller), would allow for a much smaller lower parameter on the devices housing dimensions. Furthermore, making use of more expensive components within this PCB would greatly increase the reliability and accuracy of the data gathered by wearable device. This would extend to the possibility of a more sleek and ergonomic design, maximising the user experience, increasing the likelihood that the system be implemented within more organisations.

3.3 Software

The software design for the emergency response system was the most challenging part of the design overall as there were a lot of obstacles to overcome including new coding languages (python and HTML) and difficulties including sensors and other included modules.

3.3.1 Front End

The front end of a system is the part of the system in which the customer interacts with. In this case the two aspects in which this occurs is the HTML on the pi 5, and the manual override button on the wearable itself.

3.3.1.1 HTML

The HTML website designed by our team consists of an easy to read header which shows the amount of connected devices, any active alerts, the amount of data sent over the day and the overall system runtime. This gives a brief overview for the admin staff and allows them to quickly see any active alerts to help get to the scene quicker. As we move down we see a table of all the connected devices and their most recent data. There is an option to change the refresh rate and an option to select a device to focus on a single device if necessary.

Overall the HTML turned out as we had hoped to show at the innovation fair, however leaves room for improvements as there are still some aspects which are not working. The analytics and settings buttons currently do not lead to another page and as well as the time being on a different line to the rest of the table values. If we were to continue this project, these aspects would be a priority to fix as our main goal is customer satisfaction.

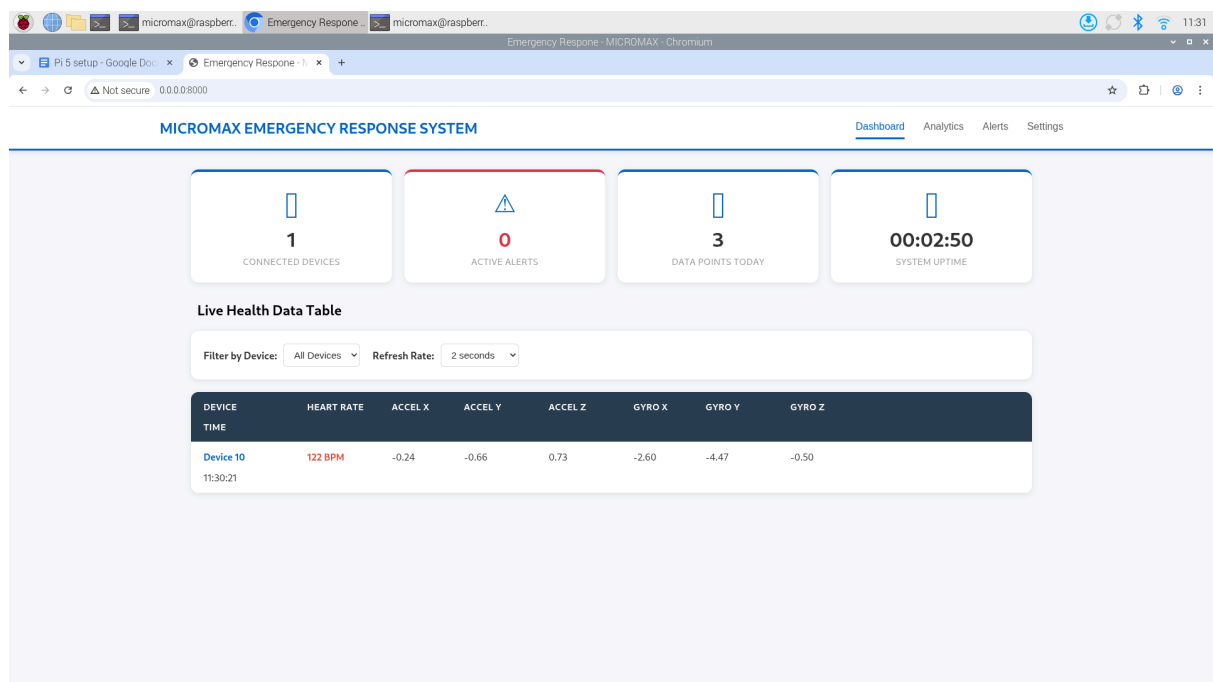


Figure 11: Emergency Response Dashboard

3.3.1.2 Emergency Override Button

While we would like to hope our product will be completely autonomous, we believe we have an ethical obligation to place a manual override on the device so that any false negatives can be avoided, and any emergency our sensors are not made to detect are still able to be alerted by employees. The manual override button is a simple addition that makes our product much safer, as if the button is held for 5 seconds, a text is sent to the onsite manager to alert them of this danger.

This simple feature makes customers feel safer with our device and if this project was to be continued there would be no need to enhance this feature further.

3.3.1.3 Server

By initialising the server on the pi 5, each pi pico acts as a client to this server. All data is sent every 10 seconds for the data to be interpreted by the pi 5, which acts as the central hub for all processing. It also routes all data to the HTML webpage for easy access to any alerts.

3.3.2 Back End

The back end of the system is not made to be handled by consumers, and shouldn't be touched at all after product rollout. There are many back end aspects to our product including all software related to the Pi Pico, the Pi 5, the eSIM, the Accelerometer / Gyroscope, heart rate sensor and GPS.

3.3.2.1 eSIM/SIM module

From the start we had issues with the SIM module we had decided to use. The SIM800L was originally chosen due to its affordability and text capabilities, however we came to realise this module was extremely old and only supported the 2G network, which is well out of date in Australia. Our second option was a SIM7070G shield made for arduino, which also had issues due to its power requirements and overall build. This led us to our final design choice, which was an eSIM approach. By utilising twilio, an online SIM service, the text implementation became a lot easier as it required no hardware, and just a free account which included \$15 trial credit, and then only \$0.0515 per text after trial.

The final design not only made the device more compact, but also more simple to set up. Twilio's low pricing per text made this option extremely viable, and we will be keeping this design element in place if the product is continued.

3.3.2.2 Accelerometer/ Gyroscope

The accelerometer and gyroscope module used in this design is a MPU-6050. This small form factor module was chosen due to its affordability and compact design, perfect for a wearable device. To make this detect falls, we took the overall magnitude of all the directions, and tested threshold values to see when a fall was detected.

In future, an AI model could perhaps be used in conjunction with the sensor to learn the regular outputs of this sensor for each individual client, so that threshold values will not need to be adjusted and a fall can be detected by abnormalities rather than a set value.

3.3.2.3 Heart Rate Sensor

The heart rate sensor chosen for this project is the MAX30102 sensor. This module does not directly measure heart rate, but rather measures the pulses, therefore a formula is needed to turn these peaks into heart rate. By taking a sampling period and measuring how many peaks were measured, we are then able to find the BPM. The only issue with this sensor is sometimes it is unable to detect a finger and therefore no reading is made.

In future prototypes, a new sensor may be chosen for greater reliability, and similarly to the MPU-6050, an AI model may be trained to understand each individual's heart rate better as all resting heart rates are different, to insure the product will reach its full potential.

3.3.2.4 GPS

The GPS module was unfortunately the only part of the initial design that we were unable to get working for our final prototype. Our initial concept utilised a U-BLOX NEO-6M V2 module which we had chosen due to its low cost. The issue with this module was that it took a very long time to initially connect to the satellites, and also needed a direct view of at least 3. This means our product which is intended for indoor use would not be able to use this module. The second approach included talking to our industry sponsor, where he recommended doing research into a BLE module. After doing this research we had found a module that had said it came with a USB, which we thought was perfect as the USB could plug into the Pi 5 and locate the module from there, however this did not come with a USB so we were unable to get it working. After this second approach we were short on time and thought the best option was to focus on the other features.

In future, a GPS will be top priority to implement, as this will allow the location to be sent directly when an emergency occurs.

3.3.2.5 Raspberry Pi Pico Code

The pi pico code includes a mixture of the accelerometer/gyroscope, heart rate module, and emergency override button. The combination of these complete the wearable device software and would include GPS module code in the future to gather location when an emergency is detected.

3.3.2.6 Raspberry Pi 5 Code

The Raspberry Pi 5 code consists of the eSIM code, the HTML code and the code for the server. By combining these, the data is sent wirelessly to the server from the pi pico, then routed to a website where the data is displayed professionally for admin purposes. The eSIM sends a text when any data meets the threshold values, or when the emergency override button is pressed for 5 seconds.

One issue that occurred when downloading libraries such as twilio and flask to set up the eSIM and server aspects was that the pi 5 declared these libraries as dangerous for the hardware. To bypass this error, as they were needed for development, a virtual environment was initialised and the files were then downloaded onto that, to keep the pi 5 safe and to ensure all aspects could be set up. This makes setting up the server slightly harder as the virtual environment must be activated before use. This activates the virtual environment called myenv and then locates the needed files called server_twilio_button.py. The steps include typing these lines to the terminal:

```
python3 -m venv myenv
source myenv/bin/activate
cd ~/pico_dashboard/templates
python3 server_twilio_button.py
```

Overall, the software components of our prototype turned out as expected for this prototype, except for the GPS module. In future, many improvements are to be made to increase latency, quality and the viability of our product in the workplace as our current rendition is just a prototype.

4.0 Project Results

Overall results for the project were a working prototype fitted inside a casing for both the wearable and central hub. At the innovation fair we plan to have 4 main components, the central hub with the HTML being displayed, a wearable device standalone, a breadboarded wearable so that the internals are able to be seen and comprehended, and a broken down casing.

There are many improvements to be made on our initial prototype such as making the wearable smaller to accommodate for the wearer, implementing GPS tracking so the emergency location can be known immediately, and just overall improvements such as finding the best threshold values or implementing other strategies to reduce false positives and false negatives. Other things to consider in the future include battery usage, as it has come to our attention that if the battery discharges slightly it may slightly affect the values being output from the sensors, and

The things that were done well throughout this project were the fall detection, by using a freefall state and then detecting impact to reduce false positives if the person moves too fast, as well as the utilisation of eSIM technology to make sure the text is sent immediately after a detected fall or abnormal heart rate. Other things that were done well was getting the server to take all values from the wearable and display them on the HTML for office use in the workplace, and display use at the innovation fair.

While room for many improvements, we think the Micromax Emergency Response System we have produced over the last year fits within the scope given to us by Shakif at Micromax and are happy with the product we are able to present at the innovation fair.

5.0 Project Management

5.1 Project Management Overview

A study conducted by Vivid Recruitment, a leading engineering company has stated that “project management ... a skill that’s increasingly essential for engineers”. This is something that resonates directly with Engineering and Design Management 3 (ECTE351). Our team has been working consistently and productively throughout the year within the ECTE351 subject on having a working device ready for the Spring Innovation Fair in Week 13 of the spring semester. It has been an immense project where all members have contributed and brought a different facet to the team. One of the hardest challenges was the communication with several members all having various schedules; this is something that was clearly evident when we completed the STAR mapping and transferable skills worksheets. Additionally, throughout the year we have utilised a multitude of project management techniques which include gantt charts, work breakdown structures, and meeting minutes all to assist with reaching our milestones and overall goal of delivering an emergency response system on behalf of Micromax.

From the audit report that our team submitted in the Spring session we reviewed the roles that we had assigned from the Autumn session. As a team we had decided that it was working well and there should be no changes. This can be seen in the table below (start of next page). Each member had an integral role that if one link was struggling the team would be there to assist and help each individual out. It was also noted that each member was not strictly set to that role and were able to float to assist one another as required.

Name	Previous role in group as at week 5 role	Current role in group as at week 5 role
Zachary Wark	Team Coordinator	Team Coordinator
Thomas Speer	Knowledge Keeper/Editor	Knowledge Keeper/Editor
Nur Azmina Mohd Rashidi	Documenter	Documenter
Oliver Michels	Lead Hardware	Lead Hardware
Ziyue Li	Lead Software	Lead Software
Laila Salem	Financial Officer/Quality Control	Financial Officer/Quality Control

Table 1: Roles of Each Team Member

5.2 Project Management Techniques

5.2.1 Gantt Chart

The gantt chart is an effective tool that our team developed early in the Autumn session and it has been ever evolving to showcase what tasks were planned and being completed. This method was great as it allowed the team to be able to forecast what tasks were scheduled, being completed or are in delay. As you can see in Appendix G the planned completion of most tasks was not necessarily always met and there were delays that our team faced. This is a common occurrence within a project and it is something that if not accounted for can be the difference between completion and failure. Again referring to the gantt chart you can see that we allowed for slack in the program to ensure that we had contingency built into the project.

5.2.2 Work Breakdown Structure

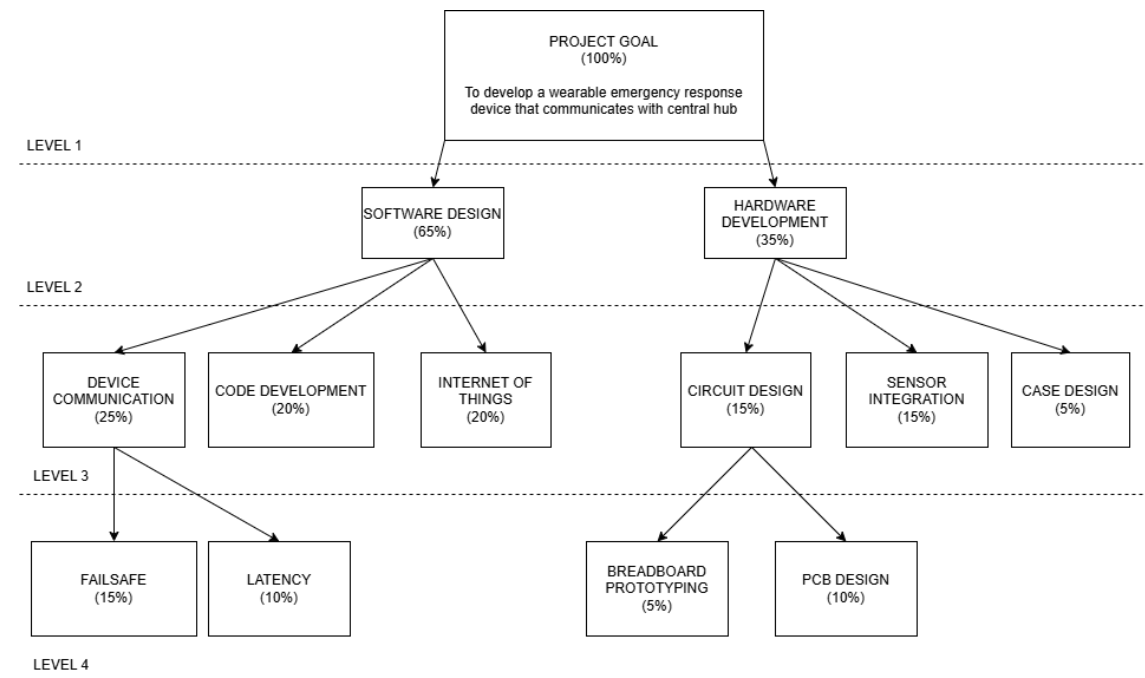


Figure 12: Developed Work Breakdown Structure

The workbreakdown structure was a great tool that we implemented in both sessions as it was able to keep the team focused on individual tasks and allow the team to operate more efficiently. This is something that was able to split our resources effectively and ensure that we would be able to meet all our requirements. One notable component that we only just fell short on without compromising on functionality was the PCB design. This identifies that we allocated our resources efficiently to still have a functional model and only dropped off a task that would have made the device more streamlined.

5.2.3 Weekly Meeting Minutes

This technique was used to document and keep records of each tutorial, workshop, and additional meeting with Shakif. It was vital that this was completed as it would ensure each member of the team was on the same page and had something to keep them accountable. Throughout any complex project this technique is applied and it allows for effective collaboration from several entities.

6.0 Budget

The University of Wollongong allotted a starting budget for the ECTE351 project of \$350, with the project's sponsor, Micromax, offering an additional budget subsidisation of up to \$1500, totalling an overall project budget of \$1850. The final cost for the project totals \$257.77 (equating to 13.93% of the total overall budget), with \$159.27 of this being subsidised by Micromax, reducing the starting budget cost to \$98.50 (equating to 28.14% of the starting budget). Throughout the project's research and development, it was decided that certain components were no longer necessary, as well as the failed integration of a BLE module.

6.1 Expenditure Record

6.1.1 Comprehensive Expenditure Record

Description	Supplier	Supplier SKU	Cost	Qty	Subsidided By Micromax	Total	Remaining Budget
Raspberry Pi Pico 2 W	Core Electronics	CE09995	\$12.48	2	No	\$24.96	\$325.04
OLED Display (SSD1306)	Core Electronics	CE09493	\$7.70	2	No	\$15.40	\$309.64
Polymer Lithium Ion Battery (LiPo) 3.7V 400mAh	Core Electronics	CE04375	\$7.15	2	No	\$14.30	\$295.34
Biometric sensor (Heart rate, Pulse oximeter)	Core Electronics	CE09443	\$4.15	2	No	\$8.30	\$287.04
MPU-6050 Module 3 Axis Gyroscope + Accelerometer	Core Electronics	018-MPU-6050	\$6.30	2	No	\$12.60	\$274.44
Mini Push Button Switch	Core Electronics	COM-00097	\$0.52	2	No	\$1.04	\$273.40
Makerverse USB-C LiPo Charger	Core Electronics	CE08374	\$5.95	2	No	\$11.90	\$261.50
U-blox NEO-6M Compatible GPS Module	Core Electronics	CE05949	\$7.50	2	No	\$15.00	\$246.50
SIM800L Text Module	Amazon	Jectseg3w0rpq1gm	\$14.69	2	No	\$29.38	\$217.12
Vero Board	Jaycar	HP9540	\$7.95	2	No	\$15.90	\$201.22
Raspberry Pi 5 Model B 8GB	Core Electronics	CE09786	\$138.20	1	Yes	\$0.00	\$201.22
Raspberry Pi 5 White Power Supply (Official) USB-C PD 5.1V 27W	Core Electronics	CE09787	\$21.07	1	Yes	\$0.00	\$201.22
NRF52840 Bluetooth Low Energy Module with USB - MDBT50Q-1M	Core Electronics	ADA4078	\$25.25	1	No	\$25.25	\$175.97
Shipping Costs							\$8.00
Total Spent							\$174.03
Final Remaining Budget							\$175.97

Table 2: Comprehensive Expenditure Record

6.1.2 Final Components List

Description	Supplier	Supplier SKU	Cost	Qty	Total
Raspberry Pi Pico 2 W	Core Electronics	CE09995	\$12.48	2	\$24.96
OLED Display (SSD1306)	Core Electronics	CE09493	\$7.70	2	\$15.40
Polymer Lithium Ion Battery (LiPo) 3.7V 400mAh	Core Electronics	CE04375	\$7.15	2	\$14.30
Biometric sensor (Heart rate, Pulse oximeter)	Core Electronics	CE09443	\$4.15	2	\$8.30
MPU-6050 Module 3 Axis Gyroscope + Accelerometer	Core Electronics	018-MPU-6050	\$6.30	2	\$12.60
Mini Push Button Switch	Core Electronics	COM-00097	\$0.52	2	\$1.04
Makerverse USB-C LiPo Charger	Core Electronics	CE08374	\$5.95	2	\$11.90
Shipping Costs					\$0.00
Total					\$88.50

Table 3: Wearable Device Final Components List

Description	Supplier	Supplier SKU	Cost	Qty	Total
Raspberry Pi 5 Model B 8GB	Core Electronics	CE09786	\$201.22	1	\$201.22
Raspberry Pi 5 White Power Supply (Official) USB-C PD 5.1V 27W	Core Electronics	CE09787	\$201.22	1	\$201.22
Shipping Costs					\$0.00
Total					\$159.27

Table 4: Central Hub Final Components List

6.2 Man Hour Expenditures

Description	Hourly Rate	Hours	Total
Consultation	\$200.00	5	\$1000
Workshop Time	\$75.00	12	\$900
Additional Team Member Time	\$100	6	\$600
Total Cost			\$2500

Table 5: Man Hour Expenditures

6.3 Cash Flow Forecasts

Taking the final components lists for the E.R.S., the cash flow forecast was calculated (See Appendix E) to result in a return on investment for each order, per number of wearable devices required as follows:

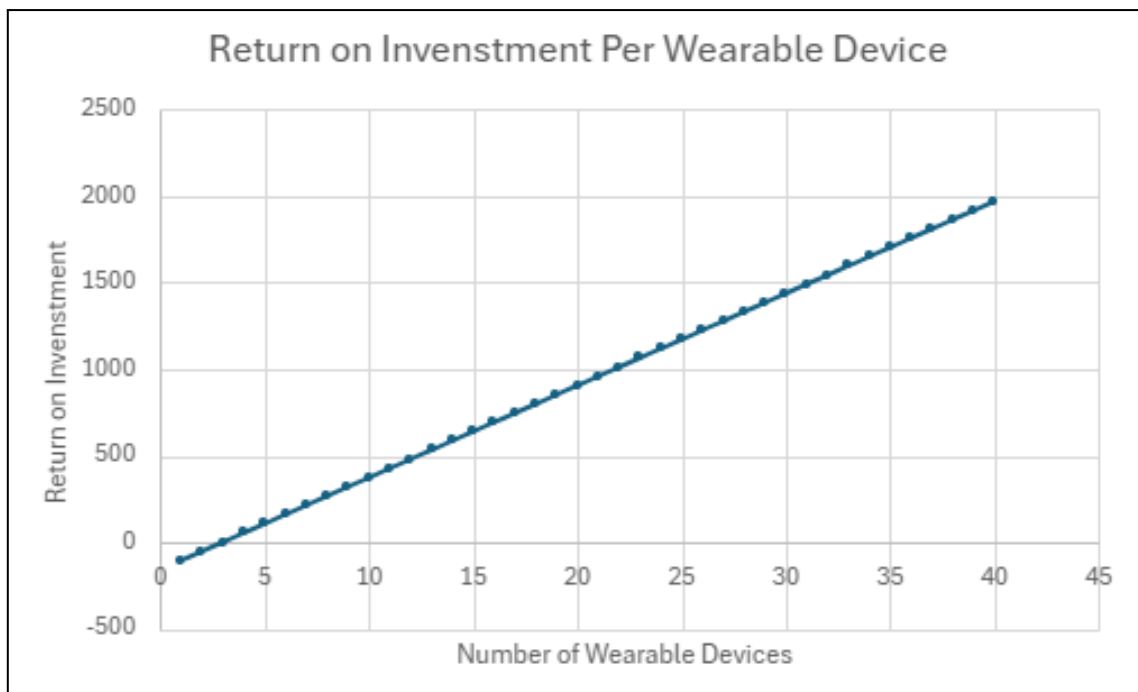


Figure 13: Return on Investment Per Wearable Device

7.0 Ethical Considerations

The Micromax Emergency Response system has been developed to enhance workplace safety. While the device has been equipped with many important features like location tracking, heart rate and motion sensors. These benefits also introduce complex ethical concerns in privacy, consent and professional accountability. In accordance with the Engineers Australia Code of Ethics [1], our team has ensured that all design and implementation decisions have been discussed with these ethical dilemmas in mind.

The first major ethical issue is in the collection and storage of personal data. The device collects highly sensitive data such as users heart rate and location which then raises concerns around informed consent and privacy. Section 1.3 of the EA Code [1] states that engineers must “respect the dignity of all persons,” the way in which the hub handles the data collected ensures information is used strictly for emergency situations. The system being used in any other way would violate section 1.2 [1] which calls for the engineers to be “honest and trustworthy”. Users are told what data is being collected, how it is being used and who has access to it for transparency reasons. As section 3.3 [1] emphasises effective and truthful communication with stakeholders, it's important to notify all personnel that there is an opt-out feature so participation remains optional without consequence. Also, the data will be retained for a short period of time before being permanently deleted as old locations and other features become useless. This is done to make sure the user's privacy is safeguarded in every way possible.

Although data privacy is critical, the system's primary ethical commitment is in the improvement of workplace safety. The main objective of the device is to help in reducing response times to incidents and increase injury report rates for accidents that usually go unnoticed. There are a few concerns with this type of technology, the first one is around false negatives. This would mean that the device doesn't trigger a real incident putting the worker at risk of delayed assistance. Another issue is around false positives, here the overuse of alarming can cause stress or ignorance to a real emergency. As section 2.3 [1] states “act on the basis of adequate knowledge” these concerns were addressed through ample testing and continuous performance monitoring. To also align with section 3.3[1] again, users are clearly informed of the system's limitations fostering an environment of accountability and honest communication.

Professional accountability is a central ethical aspect of the project. In accordance with section 2.1 [1] which says engineers should “maintain and develop knowledge and skills”, all the sensors and software have endured strict and effective testing to validate the knowledge acquired. If an error were to occur there are firm procedures for immediate documentation and remediation to ensure transparency and responsibility as stated in section 4.1 [1]. Having this system allows the engineers to uphold trust and integrity while promptly correcting any issues or concerns users or stakeholders have.

Overall, this Emergency Response System ethical foundation is based upon the Engineers Australia Code of Ethics and is guided by what our team believes is morally right. Our incorporation of privacy protection, user choice, professional accountability and transparent communication ensures that the device improves workplace safety in all the best ways. This system utilises new technologies in a way to save lives while still “respecting the dignity of all persons”, which is what our team aims to do.

8.0 Sustainability

Sustainability was a major consideration when designing the Emergency Response System. This is shown through the teams' dedication to reducing environmental impacts, improving social sustainability and ensuring there are no negative economic effects while also adhering to the United Nations Sustainable Development Goals (SGDs). The goals in which this specific system aligns with are:

- 3. Good Health and well-being; by improving workplace safety and emergency responsiveness.
- 9. Industry, Innovation and infrastructure; by promoting innovation through affordable and scalable safety technology.
- 12. Responsible Consumption and Production; by utilising recyclable materials and modular components.

8.1 Environmental sustainability

In order to reduce the system's environmental impact, low-power electronics and a rechargeable lithium battery is used. This decision will help in increasing the lifespan of the device while also reducing single use battery wastage which is quite harmful to the environment. The PLA enclosure for both the wearable device and central hub is biodegradable and recyclable, which minimises plastic waste during production. The use of a modular design allows for the individual components to be replaced rather than having to throw away the whole device. This allows for the device to be easily repaired or upgraded without unnecessary waste.

8.2 Social Sustainability

Social sustainability is obtained by improving the workers' wellbeing and safety. This system enhances the emergency response time ensuring that the incidents are reported and addressed as soon as possible. Especially in high-risk employment or settings like a warehouse, factory or when operating dangerous equipment. Since only authorised individuals have access to the stored data in the event of an emergency, protecting the users privacy is a top priority. We are also very transparent in telling the users how and what data is collected. The inclusion of alerts, accessible design features and different application styles the system fosters a nature of inclusivity and safety for all workers. As employees of all ages, abilities and backgrounds deserve to feel protected and valued by their employer.

8.3 Economic Sustainability

Choosing durable and reasonably priced components was decided upon as this guarantees affordability and long-term value. This has been proven as the anticipated cost of the final prototype shows the viability of large-scale production without sacrificing quality. This system helps organisations economically by lowering injury related expenses and increasing worker retention. This is because the device helps in speeding up emergency response times and decreases the amount of unreported incidents.

9.0 Conclusion and Recommendations

The Micromax Emergency Response System successfully met its design objectives of producing a fully functional prototype that performs real time injury detection and automated emergency communication. The combination of heart rate and motion detection, eSIM communication and live data through HTML dashboard demonstrate how wearable technology can support rapid emergency response and workplace safety. The project highlights the value of combining electronics, software and ethical design to achieve both safety and compliance goals. The use of sustainable material such as PLA for 3D printing, demonstrates environmental responsibilities and aligns with the Engineering Australia Code of Ethics in protecting community and environmental wellbeing. Despite the successful development of the prototype, there are several aspects of the design that could be refined and improved. The wearable device can be made more compact through custom PCB design, improving ergonomics and usability. The GPS module should be optimised for indoor operation, where signal interference currently limits effectiveness. Moreover, integration AI-based algorithms for adaptive data processing and more accurate event detection.

Recommendations:

1. Redesign the device using a custom PCB to make it more compact, ergonomics and durable for term use.
2. Add a reliable GPS or BLE-based tracking system to provide accurate location and consistent performance in both indoor and outdoor environment
3. Implement AI driven algorithms to increase detection accuracy, reduce false alarms and tailor sensor calibration to the behaviour patterns of individual users.
4. Conduct full compliance testing in accordance with Work Health and Safety (WHS) guidelines and Australian regulatory standards including Therapeutic Goods Administration (TGA) and Regulatory Compliance Mark (RCM) certification requirements.

Overall, the project shows strong potential for real-world application and makes valuable contributions to improve workplace safety through innovative and intelligent engineering design.

10.0 Individual Contribution

10.1 Laila Salem (7545447)

My role is group 11 has remained the same throughout the entire year, I am the financial and quality control officer. In this role I have completed many tasks including overseeing the budget, completing prototype testing to guarantee product quality and ensuring the ethical and sustainable standards given to us were upheld. These roles gave me the opportunity to collaborate on every aspect of this project.

Connecting cost-effectiveness and sustainability was a key component of my work this semester. As I have conducted a lot of research to find the most reasonably priced and reliable products. This was done as we wanted to get the most benefit out of our emergency response system without going over the budget supplied. To reduce our environmental impact, we had proposed the idea of using the recyclable PLA filament for the casing of our device and central hub. These types of decisions display responsible engineering practices and are in line with the Engineers Australia Code of Ethics.

For quality control, I make edits to the documents before they are uploaded and monitor the testing results to ensure we are consistently meeting the technical requirements of the project. I had kept track of the milestones achieved thus far to ensure that the team stays structured and on time with the Gantt chart designed. I was also tasked with documenting the problems foreseen as the prototype developed and coming up with the best solution in the most timely manner.

These roles have allowed me to learn the best way to collaborate with members of all viewpoints, standards and work styles. My experience in these roles has provided me with valuable insight into how engineering principles are combined to produce a product that is both innovative and ethical. All the skills developed throughout this subject will be continuously improved and expanded on in all my future engineering studies.

10.2 Thomas Speer (7679877)

Throughout this project my role assigned was knowledge keeper, however my main focus has been software development and circuit design. These tasks required me to wire the device, and develop the server, HTML and wearable device functions. This required major research as I was unfamiliar with python coming into this project, and lots of time spent developing and testing. Thankfully, for each sensor there were premade sensor classes and also some premade twilio eSIM development code that I was able to utilise to take some tasks off my hands.

This role gave me the opportunity to further develop my software skills and become more familiar with python and circuit design. I also learned to collaborate on a major project with engineers from different disciplines and discovered different views that all of them have, and how we can use these different views to improve upon our design. These skills I learned are those that can be carried into the workplace for further development.

10.3 Oliver Michels (7962228)

My role as the Lead Hardware Developer has remained consistent throughout the entirety of the 26 week project. In this role I have undertaken many tasks including overseeing which components are to be used within both the wearable device and central hub, contributing to the maintenance of the project budgeting, and the designing and developing the housing for both separate sub-systems. These responsibilities required a combination of technical knowledge, design consideration, and effective communication and collaboration with the rest of the project team to ensure that all hardware decisions aligned with the overall scope of the project.

A major focus of my work this semester was in assisting with the selection of final components to ensure a good balance of functionality and cost effectiveness. This process required close and effective communication with both the lead software developer and financial and quality control officer, ensuring the proper integration of the hardware with the software and data systems, to allow for reliable communication between the wearable device and central hub, and the proper maintenance of the project budget. Additionally, these decisions on the components to be used within the final design, created the requirement for the CAD designs to be adapted respectively. This involved creating multiple design iterations to improve the ergonomics, accessibility, manufacturability, and aesthetics of the housings. Careful considerations on material selection were given to ensure the sustainability of the project, ensuring that the hardware produced would align with the Engineers Australia Code of Ethics.

Working in this role has significantly developed my skills in hardware system integration, design optimisation, and technical project management. The undertaking of the project has emphasised the importance of maintaining consistent communication across disciplines and adapting designs in response to practical feedback, reinforcing my understanding of how collaborative teamwork and maintaining responsible engineering practices is essential for the delivery of an effective and ethical final product. I plan to continue building on these skills throughout my future engineering studies and professional career.

10.4 Zachary Wark (6784471)

My role in the team for the duration of the project was team coordinator. We reassessed in the spring session and had decided as a team that we were operating well therefore we made no changes. Being a team coordinator has underlined for me the complex nature of being able to resource manage whilst maintaining a sound understanding of the project. Throughout tutorials I took initiative in group discussions and a leading role in giving presentations. This is a skill that I wanted to improve as at my undergraduate job being able to present and have technical conversations is something I initially struggled with. From undertaking ECTE351 with my teammates I have improved this skill immensely and I would not have been able to do it without them.

I was in charge of implementing gantt charts and work breakdown structures. A software that I had used previously was Microsoft Project and this is something that I use in the workforce currently. This made this process seamless. I have learnt invaluable lessons with this team and I will continue these practices when I undoubtedly work in a team next.

Specifically in the Spring session we looked at the unit of Ethics and how this is a major topic within project delivery. As engineers this is something that plays a vital role that wasn't directly apparent to me until we had completed this in the tutorial. We looked at two case studies and then directly linked this with our Emergency Response System. Applying the Engineers Australia's Code of Ethics successfully broadened my knowledge and understanding. Specific topics of privacy and sustainability within project delivery were not necessarily in the forefront of my mind until undertaking these studies within ECTE351. I will continue to widen my knowledge as I branch into my undergraduate position that directly involves project delivery to ensure I use the skills that I learnt in the Ethics component.

10.5 Nur Azmina Mohd Rashidi (7754127)

Although my assigned role in the group was Documenter, I mainly supported my teammates by helping with various tasks. I also assisted with basic hardware assembly. I also contributed to parts of the project management and compliance documentation, including reviewing sections related to Engineers Australia Code of Ethics. This gave me a better understanding of how regulatory and ethical requirements influence engineering design and product development.

Working alongside the hardware and software teams helped me learn more about practical implementation and how every discipline connects within a real engineering project. Although I was not directly responsible for coding or detailed documentation, I participated actively in meetings and supported my team members whenever help was needed.

This experience has strengthened my teamwork, communication, and problem-solving skills. I have also gained more confidence in understanding hardware assembly, power connections, and compliance processes, which will be valuable in future engineering projects.

10.6 Ziyue Li

In this project, I served as the Lead Software Developer, primarily responsible for the initial planning of the system architecture and the design of the embedded software. The role sharpened my ability to translate theory into practical engineering and deepened my end-to-end skills in embedded development and system integration.

I also contributed to the market analysis section. Through in-depth research on industry trends, user needs, and privacy protection regulations, I learned how to align technical choices with real-world constraints during product development. This hands-on work experience helped me realise that technical prowess alone is insufficient; understanding markets, users, and social responsibility is equally vital. It motivated me to keep strengthening my cross-disciplinary capabilities and to create solutions that are technically robust as well as socially valuable and feasible.

11.0 References

- 1- Engineering Australia Code of Ethics. Available from:
<https://www.engineersaustralia.org.au/ethics>.
- 2- MPU6050 Accelerometer and gyroscope class and information. Available from:
https://www.i2cdevlib.com/docs/html/class_m_p_u6050.html
- 3- MAX30102 Heart Rate / SPO2 Monitor class and information. Available from:
<https://github.com/vrano714/max30102-tutorial-raspberrypi>

12.0 Appendices

12.1 Appendix A: Wearable Code (Pi Pico)

```

import network
import socket
import json
import time
from machine import Pin, I2C
import _thread
import math
time.sleep(1)

# wifi config
WIFI_SSID = "ENTER_YOUR_WIFI_SSID"
WIFI_PASSWORD = "ENTER_WIFI_PASSWORD"
PI5_IP = "ENTER_PI5_IP" #hostname -I (terminal pi5)
PI5_PORT = 8080

# fall thresholds
FALL_THRESHOLD_LOW = 0.4 # g - freefall detection (below this = freefall)
FALL_THRESHOLD_HIGH = 2.5 # g - impact detection (above this = impact)
FALL_MIN_DURATION = 50 # ms - minimum freefall time
FALL_MAX_DURATION = 1500 # ms - maximum freefall time

#manual button config

BUTTON_PIN = 15
BUTTON_HOLD_TIME = 1

# varisbles
latest_accel = {"x": 0, "y": 0, "z": 0}
latest_gyro = {"x": 0, "y": 0, "z": 0}
latest_heart_rate = 0
fall_detected_flag = False
data_lock = _thread.allocate_lock()

# class gyro imported from github
class MPU6050:
    def __init__(self, i2c, addr=0x68):
        self.i2c = i2c
        self.addr = addr
        self.i2c.writeto_mem(self.addr, 0x6B, b'\x00')

    def read_raw(self, reg):
        data = self.i2c.readfrom_mem(self.addr, reg, 2)
        value = int.from_bytes(data, 'big')
        if value > 32767:
            value -= 65536
        return value

    def get_accel(self):
        ax = self.read_raw(0x3B) / 16384.0
        ay = self.read_raw(0x3D) / 16384.0
        az = self.read_raw(0x3F) / 16384.0
        return ax, ay, az

    def get_gyro(self):
        gx = self.read_raw(0x43) / 131.0
        gy = self.read_raw(0x45) / 131.0
        gz = self.read_raw(0x47) / 131.0

```

```

    return gx, gy, gz

#class max30102 imported from github
REG_INTR_STATUS_1 = 0x00
REG_INTR_ENABLE_1 = 0x02
REG_FIFO_WR_PTR = 0x04
REG_OVF_COUNTER = 0x05
REG_FIFO_RD_PTR = 0x06
REG_FIFO_DATA = 0x07
REG_FIFO_CONFIG = 0x08
REG_MODE_CONFIG = 0x09
REG_SPO2_CONFIG = 0x0A
REG_LED1_PA = 0x0C
REG_LED2_PA = 0x0D
REG_PART_ID = 0xFF

class MAX30102:
    def __init__(self, i2c, address=0x57):
        self.i2c = i2c
        self.address = address
        part_id = self.i2c.readfrom_mem(self.address, REG_PART_ID, 1)[0]
        if part_id != 0x15:
            raise Exception("MAX30102 not found, check wiring!")
        self.reset()
        self.setup()

    def reset(self):
        self.i2c.writeto_mem(self.address, REG_MODE_CONFIG, b'\x40')
        time.sleep_ms(100)

    def setup(self):
        self.i2c.writeto_mem(self.address, REG_INTR_ENABLE_1, b'\xc0')
        self.i2c.writeto_mem(self.address, REG_FIFO_CONFIG, b'\x4f')
        self.i2c.writeto_mem(self.address, REG_MODE_CONFIG, b'\x03')
        self.i2c.writeto_mem(self.address, REG_SPO2_CONFIG, b'\x27')
        self.i2c.writeto_mem(self.address, REG_LED1_PA, b'\x24')
        self.i2c.writeto_mem(self.address, REG_LED2_PA, b'\x24')

    def read_fifo(self):
        data = self.i2c.readfrom_mem(self.address, REG_FIFO_DATA, 6)
        red = (data[0] << 16 | data[1] << 8 | data[2]) & 0x3FFFF
        ir = (data[3] << 16 | data[4] << 8 | data[5]) & 0x3FFFF
        return red, ir

    def get_bpm(self, sample_time=10, fs=100):
        ir_values = []
        self.i2c.writeto_mem(self.address, REG_FIFO_WR_PTR, b'\x00')
        self.i2c.writeto_mem(self.address, REG_OVF_COUNTER, b'\x00')
        self.i2c.writeto_mem(self.address, REG_FIFO_RD_PTR, b'\x00')
        t0 = time.time()
        while time.time() - t0 < sample_time * 1000:
            _, ir = self.read_fifo()
            ir_values.append(ir)
            time.sleep_ms(int(1000 / fs))

        if len(ir_values) < 2:
            return 0

        threshold = sum(ir_values) / len(ir_values) * 1.2
        peaks = []

```

```

for i in range(1, len(ir_values) - 1):
    if ir_values[i] > threshold and ir_values[i] > ir_values[i-1] and ir_values[i] > ir_values[i+1]:
        peaks.append(i)
if len(peaks) < 2:
    return 0

intervals = [(peaks[i+1] - peaks[i]) / fs for i in range(len(peaks)-1)]
avg_interval = sum(intervals) / len(intervals)
bpm = 6 / avg_interval
return int(bpm)

# class button imported from github
class EmergencyButton:
    def __init__(self, pin_number):
        self.button = Pin(pin_number, Pin.IN, Pin.PULL_UP)
        self.press_start_time = None
        self.emergency_triggered = False
        self.last_check = time.ticks_ms()

    def check(self):
        current_time = time.ticks_ms()

        if time.ticks_diff(current_time, self.last_check) < 100:
            return False

        self.last_check = current_time

        if self.button.value() == 0:
            if self.press_start_time is None:
                self.press_start_time = current_time
                print("Button pressed - hold for 3 seconds")
            else:
                press_duration = time.ticks_diff(current_time, self.press_start_time) / 1000

                if int(press_duration) > 0 and int(press_duration) % 1 == 0:
                    remaining = BUTTON_HOLD_TIME - int(press_duration)
                    if remaining > 0:
                        print(f"Hold for {remaining} more seconds...")

                if press_duration >= BUTTON_HOLD_TIME and not self.emergency_triggered:
                    self.emergency_triggered = True
                    print("EMERGENCY ALERT TRIGGERED!")
                    return True
            else:
                if self.press_start_time is not None and not self.emergency_triggered:
                    print("Button released")
                self.press_start_time = None
                self.emergency_triggered = False

        return False

    def reset(self):
        self.press_start_time = None
        self.emergency_triggered = False

# fall detect - 100ms
def fall_detection_thread():
    """Background thread that continuously monitors for falls"""
    global fall_detected_flag, latest_accel, latest_gyro

```

```

print("[Fall Detection] Thread started")

fall_state = "normal" # 3 states: normal, freefall, cooldown
# only detect fall when freefall enabled then impact detected
freefall_start = 0
error_count = 0

while True:
    try:
        # read accelerometer
        ax, ay, az = mpu.get_accel()
        gx, gy, gz = mpu.get_gyro()

        # calculate total acceleration
        total_accel = math.sqrt(ax**2 + ay**2 + az**2) # magnitude of accel
        with data_lock:
            latest_accel = {"x": round(ax, 2), "y": round(ay, 2), "z": round(az, 2)}
            latest_gyro = {"x": round(gx, 2), "y": round(gy, 2), "z": round(gz, 2)}

        error_count = 0

        # fall detection state
        if fall_state == "normal":
            # check for freefall (low acceleration)
            if total_accel < FALL_THRESHOLD_LOW:
                fall_state = "freefall"
                freefall_start = time.time()
                print(f"[FALL] Freefall detected! ({total_accel:.2f}g)")

            elif fall_state == "freefall":
                freefall_duration = time.time() - freefall_start

                # check for impact (high acceleration)
                if total_accel > FALL_THRESHOLD_HIGH:
                    # check if freefall duration was realistic
                    if FALL_MIN_DURATION < freefall_duration < FALL_MAX_DURATION:
                        print(f"[FALL] IMPACT DETECTED! {total_accel:.2f}g after {freefall_duration}ms")
                        with data_lock:
                            fall_detected_flag = True
                            fall_state = "cooldown"
                        else:
                            print(f"[FALL] False alarm (duration: {freefall_duration}ms)")
                            fall_state = "normal"

                    # timeout if there is no impact after freefall, most likely just fast movement
                    elif freefall_duration > FALL_MAX_DURATION:
                        print("[FALL] Timeout, no impact")
                        fall_state = "normal"

            elif fall_state == "cooldown":
                # wait 10 seconds before detecting again
                if time.time() - freefall_start > 10000:
                    fall_state = "normal"
                    print("[FALL] Ready for next detection")

        # check 10x per second
        time.sleep(0.1)

    except OSError as e:
        error_count += 1

```

```

        if error_count % 50 == 1:
            print(f"[Fall Detection] I2C error")
            time.sleep_ms(200)

    except Exception as e:
        print(f"[Fall Detection] ")
        time.sleep(1)

# wifi

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)

    print("Connecting to WiFi...")
    timeout = 10
    while timeout > 0:
        if wlan.status() < 0 or wlan.status() >= 3:
            break
        timeout -= 1
        print('Waiting for connection...')
        time.sleep(1)

    if wlan.status() != 3:
        raise RuntimeError('WiFi connection failed')
    else:
        print('Connected')
        status = wlan.ifconfig()
        print(f'IP address: {status[0]}')
        return wlan

def send_data(data):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(10)

        s.connect((PI5_IP, PI5_PORT))

        json_data = json.dumps(data)
        bytes_sent = s.send(json_data.encode('utf-8'))

        s.close()
        return True

    except OSError as e:
        error_str = str(e)
        if "ETIMEDOUT" in error_str:
            print("Network timeout")
        elif "ECONNREFUSED" in error_str:
            print("Connection refused")
        else:
            print(f"Network error: {e}")
        return False

    except Exception as e:
        print(f"Send error: {e}")
        return False

# main
led = Pin("LED", Pin.OUT)

```

```

led.value(1)

# connect
try:
    wlan = connect_wifi()
except Exception as e:
    print(f"WiFi failed: {e}")

# sensor initialiose
i2c0 = I2C(0, scl=Pin(1), sda=Pin(0), freq=400000)
i2c1 = I2C(1, scl=Pin(11), sda=Pin(10), freq=400000)

mpu = MPU6050(i2c0)
print("MPU6050 initialized")

max_sensor = MAX30102(i2c1)
print("MAX30102 initialized")

emergency_button = EmergencyButton(BUTTON_PIN)
print(f"Emergency button on GP15")

# start fall detection
_thread.start_new_thread(fall_detection_thread, ())
print("Fall detection thread started")

print("Ready")
time.sleep(2)

while True:
    try:
        # check for fall
        fall_alert = False
        with data_lock:
            if fall_detected_flag:
                fall_alert = True
                fall_detected_flag = False

        # sfall alert to pi5 (not sms)
        if fall_alert:
            print("SENDING FALL ALERT")

            with data_lock:
                accel = latest_accel.copy()
                gyro = latest_gyro.copy()

            fall_data = {
                "timestamp": time.ticks_ms(),
                "accelerometer": accel,
                "gyroscope": gyro,
                "heart_rate": latest_heart_rate if latest_heart_rate > 0 else None,
                "emergency": True,
                "emergency_message": "FALL DETECTED"
            }

            if send_data(fall_data):
                print("Fall alert sent!")
                time.sleep(2)

        # check emergency button
        if emergency_button.check():

```

```

print("Sending manual emergency...")

with data_lock:
    accel = latest_accel.copy()
    gyro = latest_gyro.copy()

try:
    bpm = max_sensor.get_bpm(sample_time=3)
except:
    bpm = latest_heart_rate

emergency_data = {
    "timestamp": time.time(),
    "accelerometer": accel,
    "gyroscope": gyro,
    "heart_rate": bpm if bpm > 0 else None,
    "emergency": True,
    "emergency_message": "MANUAL EMERGENCY BUTTON"
}

send_data(emergency_data)
emergency_button.reset()
time.sleep(1)
continue

# heart rate measurement
print("Measuring heart rate...")
bpm = max_sensor.get_bpm(sample_time=10)
latest_heart_rate = bpm
with data_lock:
    accel = latest_accel.copy()
    gyro = latest_gyro.copy()

sensor_data = {
    "timestamp": time.time(),
    "accelerometer": accel,
    "gyroscope": gyro,
    "heart_rate": bpm if bpm > 0 else None,
    "emergency": False
}

print(f"HR: {bpm} BPM" if bpm > 0 else "No finger")
print(f"Accel: {accel['x']:.2f}, {accel['y']:.2f}, {accel['z']:.2f} ({math.sqrt(accel['x']**2 + accel['y']**2 + accel['z']**2):.2f}g)")
print(f"Gyro: {gyro['x']:.2f}, {gyro['y']:.2f}, {gyro['z']:.2f}")

if send_data(sensor_data):
    print("Data sent")
    print("-" * 50)
    time.sleep(2)

except Exception as e:
    print(f"Main error: {e}")
    time.sleep(5)

```

12.2 Appendix B: Central Hub Code (Pi 5)

```

import socket
import json
import threading
import time
from datetime import datetime
import csv
import os
from http.server import HTTPServer, BaseHTTPRequestHandler
from urllib.parse import urlparse, parse_qs
import math

# twilio config imported
try:
    from twilio.rest import Client
    TWILIO_AVAILABLE = True
except ImportError:
    TWILIO_AVAILABLE = False
    print("WARNING: Twilio not installed. Run: pip3 install twilio")

#replace with own account details
TWILIO_ACCOUNT_SID = "AC199e308cc25f9d14c424e8cf2f98ae09"
TWILIO_AUTH_TOKEN = "fe86ba149f78b83527b319ebd0d63585"
TWILIO_PHONE_NUMBER = "+15075015730"
ALERT_PHONE_NUMBERS = ["+61401421279"]

# thresholds - change for ensitivty to alerts
ALERT_THRESHOLDS = {
    "high_heart_rate": 120,    # BPM
    "low_heart_rate": 50,     # BPM
    "max_acceleration": 3.0,  # g (for fall detection)
    "max_gyro_rotation": 180.0, # deg/s (for fall detection)
}

SENSOR_HOST = '0.0.0.0'
SENSOR_PORT = 8080
WEB_HOST = '0.0.0.0'
WEB_PORT = 8000
LOG_TO_FILE = True

#multile pico handler imported
pico_data = {}
pico_lock = threading.Lock()
alerts_list = []
MAX_HISTORY_PER_PICO = 100
MAX_ALERTS = 50
#twilio imported
class AlertSystem:
    def __init__(self):
        self.twilio_client = None
        self.alert_cooldown = {}

    if TWILIO_AVAILABLE and TWILIO_ACCOUNT_SID != "your_account_sid":

```

```

    try:
        self.twilio_client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
        print("Twilio client initialized")
    except Exception as e:
        print(f"Twilio initialization failed: {e}")
    else:
        print("Twilio not configured - alerts will be logged but not sent via SMS")

def send_sms_alert(self, pico_id, alert_message):
    """Send SMS alert via Twilio"""
    if not self.twilio_client:
        print(f"[ALERT - No SMS] Pico {pico_id}: {alert_message}")
        return False

    cooldown_key = f"{pico_id}_{alert_message[:20]}"
    if cooldown_key in self.alert_cooldown:
        if time.time() - self.alert_cooldown[cooldown_key] < 60:
            return False

    try:
        for phone in ALERT_PHONE_NUMBERS:
            message = self.twilio_client.messages.create(
                body=f"HEALTH ALERT \nDevice {pico_id}: \n{alert_message}",
                from_=TWILIO_PHONE_NUMBER,
                to=phone
            )

            self.alert_cooldown[cooldown_key] = time.time()
            print(f"SMS Alert sent for Pico {pico_id}")
            return True

    except Exception as e:
        print(f"Failed to send SMS: {e}")
        return False

def log_alert(self, pico_id, alert_type, alert_message, data):
    """Log alert to system"""
    alert_entry = {
        "timestamp": datetime.now().isoformat(),
        "pico_id": pico_id,
        "type": alert_type,
        "message": alert_message,
        "data": data
    }

    alerts_list.append(alert_entry)
    if len(alerts_list) > MAX_ALERTS:
        alerts_list.pop(0)

    print(f"[{datetime.now().strftime('%H:%M:%S')}] ALERT - Pico {pico_id}: {alert_message}")

alert_system = AlertSystem()

# fall detection
def detect_fall(accel, gyro, pico_id):

```

```

"""
Detect falls using accelerometer and gyroscope data
Returns: (is_fall, reason)
"""
if not accel or not gyro:
    return False, None

ax = accel.get('x', 0)
ay = accel.get('y', 0)
az = accel.get('z', 0)

gx = gyro.get('x', 0)
gy = gyro.get('y', 0)
gz = gyro.get('z', 0)

# calculate total acceleration (magnitude)
total_accel = math.sqrt(ax**2 + ay**2 + az**2)

# calculate total rotation (magnitude)
total_gyro = math.sqrt(gx**2 + gy**2 + gz**2)

# sudden high acceleration (free fall)
if total_accel > ALERT_THRESHOLDS["max_acceleration"]:
    return True, f"High acceleration detected: {total_accel:.2f}g"

# high rotation combined with acceleration
if total_gyro > ALERT_THRESHOLDS["max_gyro_rotation"] and total_accel > 1.5:
    return True, f"Rapid rotation + acceleration: {total_gyro:.1f}°/s, {total_accel:.2f}g"

return False, None

class DataLogger:
    def __init__(self, pico_id, filename=None):
        self.pico_id = pico_id
        if filename is None:
            self.filename =
f'sensor_data_pico_{pico_id}_{datetime.now().strftime("%Y%m%d_%H%M%S")}.csv"
        else:
            self.filename = filename
        self.setup_csv()

    def setup_csv(self):
        """Create CSV file with headers if it doesn't exist"""
        if not os.path.exists(self.filename):
            with open(self.filename, 'w', newline='') as csvfile:
                fieldnames = [
                    'timestamp', 'datetime', 'pico_id',
                    'accel_x', 'accel_y', 'accel_z',
                    'gyro_x', 'gyro_y', 'gyro_z',
                    'heart_rate', 'fall_detected', 'alert'
                ]
                writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
                writer.writeheader()

    def log_data(self, data, fall_detected=False, alert_message=""):

```

```

"""Log sensor data to CSV file"""
try:
    with open(self.filename, 'a', newline=") as csvfile:
        fieldnames = [
            'timestamp', 'datetime', 'pico_id',
            'accel_x', 'accel_y', 'accel_z',
            'gyro_x', 'gyro_y', 'gyro_z',
            'heart_rate', 'fall_detected', 'alert'
        ]
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

        row = {
            'timestamp': data.get('timestamp', ""),
            'datetime': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            'pico_id': self.pico_id,
            'accel_x': data.get('accelerometer', {}).get('x', ""),
            'accel_y': data.get('accelerometer', {}).get('y', ""),
            'accel_z': data.get('accelerometer', {}).get('z', ""),
            'gyro_x': data.get('gyroscope', {}).get('x', ""),
            'gyro_y': data.get('gyroscope', {}).get('y', ""),
            'gyro_z': data.get('gyroscope', {}).get('z', ""),
            'heart_rate': data.get('heart_rate', ""),
            'fall_detected': 'YES' if fall_detected else 'NO',
            'alert': alert_message
        }
        writer.writerow(row)

except Exception as e:
    print(f"Error logging to CSV: {e}")

# server
class HealthMonitorHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed_path = urlparse(self.path)

        # API endpoints
        if parsed_path.path == '/api/current':
            self.serve_current_data()
        elif parsed_path.path == '/api/pico':
            self.serve_pico_data(parsed_path)
        elif parsed_path.path == '/api/all':
            self.serve_all_data()
        elif parsed_path.path == '/api/history':
            self.serve_historical_data()
        elif parsed_path.path == '/api/status':
            self.serve_status()
        elif parsed_path.path == '/api/alerts':
            self.serve_alerts()
        elif parsed_path.path == '/' or parsed_path.path == '/index.html':
            self.serve_html_file()
        else:
            self.send_error(404, "File not found")

    def serve_current_data(self):
        """Serve current sensor data from first Pico (for backwards compatibility)"""

```

```

self.send_response(200)
self.send_header('Content-type', 'application/json')
self.send_header('Access-Control-Allow-Origin', '*')
self.end_headers()

with pico_lock:
    if pico_data:
        first_pico = list(pico_data.keys())[0]
        response = pico_data[first_pico]['current']
    else:
        response = {
            "timestamp": None,
            "accelerometer": {"x": 0, "y": 0, "z": 0},
            "gyroscope": {"x": 0, "y": 0, "z": 0},
            "heart_rate": None,
            "last_updated": None,
            "pico_id": None
        }

self.wfile.write(json.dumps(response).encode())

def serve_pico_data(self, parsed_path):
    """Serve data from a specific Pico"""
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.send_header('Access-Control-Allow-Origin', '*')
    self.end_headers()

    query_params = parse_qs(parsed_path.query)
    pico_id = query_params.get('id', ['1'])[0]

    with pico_lock:
        if pico_id in pico_data:
            response = pico_data[pico_id]['current']
        else:
            response = {"error": f"Pico {pico_id} not found"}

    self.wfile.write(json.dumps(response).encode())

def serve_all_data(self):
    """Serve data from all connected Picos"""
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.send_header('Access-Control-Allow-Origin', '*')
    self.end_headers()

    with pico_lock:
        all_data = {}
        for pico_id, pico_info in pico_data.items():
            all_data[pico_id] = pico_info['current']

    self.wfile.write(json.dumps(all_data).encode())

def serve_historical_data(self):
    """Serve historical data for charts"""

```

```

self.send_response(200)
self.send_header('Content-type', 'application/json')
self.send_header('Access-Control-Allow-Origin', '*')
self.end_headers()

query_params = parse_qs(urlparse(self.path).query)
pico_id = query_params.get('id', ['1'])[0]
limit = int(query_params.get('limit', [50])[0])

with pico_lock:
    if pico_id in pico_data:
        history = pico_data[pico_id]['history']
        data_to_send = history[-limit:] if len(history) > limit else history
    else:
        data_to_send = []

self.wfile.write(json.dumps(data_to_send).encode())

def serve_alerts(self):
    """Serve recent alerts"""
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.send_header('Access-Control-Allow-Origin', '*')
    self.end_headers()

    self.wfile.write(json.dumps(alerts_list).encode())

def serve_status(self):
    """Serve system status"""
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.send_header('Access-Control-Allow-Origin', '*')
    self.end_headers()

    with pico_lock:
        total_readings = sum(len(pico_info['history']) for pico_info in pico_data.values())
        connected_picos = list(pico_data.keys())

    status = {
        "server_time": datetime.now().isoformat(),
        "total_readings": total_readings,
        "connected_picos": connected_picos,
        "pico_count": len(pico_data),
        "data_available": len(pico_data) > 0,
        "alert_count": len(alerts_list),
        "twilio_enabled": alert_system.twilio_client is not None
    }

    self.wfile.write(json.dumps(status).encode())

def serve_html_file(self):
    """Serve the health monitoring HTML file"""
    try:
        html_files = ['health_monitor.html', 'index.html', 'monitor.html']
        html_file = None

```

```

for filename in html_files:
    if os.path.exists(filename):
        html_file = filename
        break

if html_file:
    with open(html_file, 'r') as f:
        content = f.read()

    self.send_response(200)
    self.send_header('Content-type', 'text/html')
    self.end_headers()
    self.wfile.write(content.encode())
else:
    self.serve_basic_page()

except Exception as e:
    print(f"Error serving HTML: {e}")
    self.serve_basic_page()

def serve_basic_page(self):
    """Serve a basic HTML page if the main HTML file isn't found"""
    html_content = """
    <!DOCTYPE html>
    <html>
    <head>
        <title>Health Monitor</title>
        <style>
            body { font-family: Arial, sans-serif; margin: 40px; }
            .status { background: #f0f0f0; padding: 20px; border-radius: 8px; }
        </style>
    </head>
    <body>
        <h1>Health Monitor Server</h1>
        <div class="status">
            <h2>Server Running</h2>
            <p>The health monitor server is running, but the HTML dashboard file wasn't found.</p>

            <h3>Available API Endpoints:</h3>
            <ul>
                <li>/api/current - Current data from first Pico</li>
                <li>/api/all - Current data from all Picos</li>
                <li>/api/alerts - Recent alerts</li>
                <li>/api/status - Server status</li>
            </ul>
        </div>
    </body>
    </html>
    """

    self.send_response(200)
    self.send_header('Content-type', 'text/html')
    self.end_headers()
    self.wfile.write(html_content.encode())

```

```

def log_message(self, format, *args):
    """Suppress default logging"""
    pass

def handle_sensor_client(conn, addr, logger_dict):
    """Handle incoming sensor data from Pico"""
    global pico_data

    client_id = f"{addr[0]}:{addr[1]}"
    print(f"\n[{datetime.now().strftime('%H:%M:%S')}] New connection from {client_id}")

    try:
        # receive data from pico imported
        data = conn.recv(1024)
        if data:

            json_str = data.decode('utf-8')
            sensor_data = json.loads(json_str)

            pico_id = addr[0].split('.')[-1] #pico identifier id

            with pico_lock:
                if pico_id not in pico_data:
                    pico_data[pico_id] = {
                        'current': {},
                        'history': [],
                        'last_ip': addr[0],
                        'logger': None
                    }

                if LOG_TO_FILE:
                    logger = DataLogger(pico_id)
                    pico_data[pico_id]['logger'] = logger
                    logger_dict[pico_id] = logger

            pico_data[pico_id]['current'] = sensor_data
            pico_data[pico_id]['current']['last_updated'] = datetime.now().isoformat()
            pico_data[pico_id]['current']['pico_id'] = pico_id

            historical_entry = {**sensor_data, "datetime": datetime.now().isoformat()}
            pico_data[pico_id]['history'].append(historical_entry)

            if len(pico_data[pico_id]['history']) > MAX_HISTORY_PER_PICO:
                pico_data[pico_id]['history'] =
pico_data[pico_id]['history'][-MAX_HISTORY_PER_PICO:]

            print(f"[Pico {pico_id}] Data received:")

            # show data
            accel = sensor_data.get('accelerometer', {})
            print(f" Accelerometer [g]: X={accel.get('x', 'N/A')}, Y={accel.get('y', 'N/A')},
Z={accel.get('z', 'N/A')}")

            gyro = sensor_data.get('gyroscope', {})

```

```

        print(f" Gyroscope [dps]: X={gyro.get('x', 'N/A')}, Y={gyro.get('y', 'N/A')}, Z={gyro.get('z',
'N/A')}")

        heart_rate = sensor_data.get('heart_rate')
        print(f" Heart Rate: {heart_rate} BPM" if heart_rate else " Heart Rate: No finger detected")

        #checking fir alerts
        alerts_triggered = []

        # button check
        if sensor_data.get('emergency', False):
            emergency_msg = sensor_data.get('emergency_message', 'EMERGENCY BUTTON
ACTIVATED')
            alert_msg = f"EMERGENCY: {emergency_msg}"
            alerts_triggered.append(("emergency", alert_msg))
            print(f"EMERGENCY BUTTON PRESSED!")

        # threshold check
        if heart_rate and heart_rate > ALERT_THRESHOLDS["high_heart_rate"]:
            alert_msg = f"HIGH HEART RATE: {heart_rate} BPM (Threshold:
>{ALERT_THRESHOLDS['high_heart_rate']})"
            alerts_triggered.append(("high_hr", alert_msg))

        if heart_rate and heart_rate < ALERT_THRESHOLDS["low_heart_rate"]:
            alert_msg = f"LOW HEART RATE: {heart_rate} BPM (Threshold:
<{ALERT_THRESHOLDS['low_heart_rate']})"
            alerts_triggered.append(("low_hr", alert_msg))

        is_fall, fall_reason = detect_fall(accel, gyro, pico_id)
        if is_fall:
            alert_msg = f"POSSIBLE FALL DETECTED: {fall_reason}"
            alerts_triggered.append(("fall", alert_msg))

        #text send
        for alert_type, alert_msg in alerts_triggered:
            alert_system.log_alert(pico_id, alert_type, alert_msg, sensor_data)
            alert_system.send_sms_alert(pico_id, alert_msg)

        if LOG_TO_FILE and pico_id in logger_dict:
            alert_str = " | ".join([msg for _, msg in alerts_triggered]) if alerts_triggered else ""
            logger_dict[pico_id].log_data(sensor_data, fall_detected=is_fall, alert_message=alert_str)

        print(f" Total Picos connected: {len(pico_data)}")
        if alerts_triggered:
            print(f" ⚠️ ALERTS: {len(alerts_triggered)} triggered")
            print("-" * 60)

    except json.JSONDecodeError as e:
        print(f"Error decoding JSON: {e}")
    except Exception as e:
        print(f"Error handling sensor data: {e}")

    finally:
        conn.close()

```

```

def start_sensor_server():
    """Start the TCP server to receive data from Pico"""
    logger_dict = {}
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        server_socket.bind((SENSOR_HOST, SENSOR_PORT))
        server_socket.listen(5)
        print(f"Sensor server listening on {SENSOR_HOST}:{SENSOR_PORT}")
        print("Waiting for Pico connections...")

        while True:
            conn, addr = server_socket.accept()
            client_thread = threading.Thread(
                target=handle_sensor_client,
                args=(conn, addr, logger_dict)
            )
            client_thread.daemon = True
            client_thread.start()

    except Exception as e:
        print(f"Sensor server error: {e}")
    finally:
        server_socket.close()

def start_web_server():
    """Start the web server"""
    try:
        server = HTTPServer((WEB_HOST, WEB_PORT), HealthMonitorHandler)
        print(f"Web server starting on http://{WEB_HOST}:{WEB_PORT}")
        server.serve_forever()
    except Exception as e:
        print(f"Web server error: {e}")

# main
if __name__ == "__main__":
    print("Multi-Pico Health Monitor Server with Alerts")
    print("=" * 50)
    print(f"Sensor port: {SENSOR_PORT}")
    print(f"Web port: {WEB_PORT}")
    print(f"Twilio Status: {'ENABLED' if alert_system.twilio_client else 'DISABLED'}")
    print("Alert Thresholds:")
    for key, value in ALERT_THRESHOLDS.items():
        print(f"  {key}: {value}")
    print("=" * 50)

    try:
        sensor_thread = threading.Thread(target=start_sensor_server)
        sensor_thread.daemon = True
        sensor_thread.start()

        start_web_server()

    except KeyboardInterrupt:

```

```

    print("\nShutting down servers...")
except Exception as e:
    print(f"Server startup error: {e}")

```

12.3 HTML Code

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Emergency Response - MICROMAX </title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell,
sans-serif;
      background: #f5f7fa;
      color: #333;
    }

    /* Header */
    .header {
      background: white;
      border-bottom: 3px solid #0066cc;
      padding: 0;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    }

    .header-content {
      max-width: 1500px;
      margin: 0 auto;
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 15px 30px;
    }

    .logo {
      display: flex;
      align-items: center;
      gap: 10px;
      font-size: 1.5rem;
      font-weight: 700;
      color: #0066cc;
    }

    .logo-icon {
      font-size: 1.8rem;
    }

    .nav-tabs {
      display: flex;
      gap: 30px;
    }

```

```

.nav-tab {
  padding: 8px 0;
  border: none;
  background: none;
  cursor: pointer;
  font-size: 1rem;
  color: #666;
  border-bottom: 3px solid transparent;
  transition: all 0.3s ease;
}

.nav-tab.active {
  color: #0066cc;
  border-bottom-color: #0066cc;
}

.nav-tab:hover {
  color: #0066cc;
}

/* Main Container */
.container {
  max-width: 1400px;
  margin: 0 auto;
  padding: 30px;
}

/* Stat Cards */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}

.stat-card {
  background: white;
  border-radius: 12px;
  padding: 25px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  text-align: center;
  border-top: 4px solid #0066cc;
}

.stat-icon {
  font-size: 2.5rem;
  margin-bottom: 15px;
  color: #0066cc;
}

.stat-value {
  font-size: 2rem;
  font-weight: 700;
  color: #333;
  margin-bottom: 8px;
}

.stat-label {
  font-size: 0.9rem;
  color: #999;
}

```

```

    text-transform: uppercase;
    letter-spacing: 0.5px;
    font-weight: 500;
}

/* Section Title */
.section-title {
    font-size: 1.4rem;
    font-weight: 700;
    color: black;
    margin-bottom: 20px;
    display: flex;
    align-items: center;
    gap: 10px;
}

.section-icon {
    font-size: 1.5rem;
}

/* Controls */
.controls {
    background: white;
    border-radius: 12px;
    padding: 20px;
    margin-bottom: 20px;
    display: flex;
    gap: 20px;
    align-items: center;
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    flex-wrap: wrap;
}

.control-group {
    display: flex;
    align-items: center;
    gap: 10px;
}

.control-label {
    font-weight: 600;
    color: #333;
}

select {
    padding: 8px 12px;
    border: 1px solid #ddd;
    border-radius: 6px;
    background: white;
    cursor: pointer;
    font-size: 0.95rem;
    color: #333;
}

/* Data Table */
.data-table-container {
    background: white;
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);

```

```

    margin-bottom: 30px;
}

.table-header {
    background: #2c3e50;
    color: white;
    padding: 15px 20px;
    display: grid;
    grid-template-columns: 150px 120px 100px 120px 120px 100px 120px 150px;
    gap: 15px;
    font-weight: 600;
    font-size: 0.9rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.table-header-item {
    display: flex;
    align-items: center;
    gap: 5px;
}

.table-body {
    max-height: 400px;
    overflow-y: auto;
}

.table-row {
    display: grid;
    grid-template-columns: 150px 120px 100px 120px 120px 100px 120px 150px;
    gap: 15px;
    padding: 15px 20px;
    border-bottom: 1px solid #eee;
    align-items: center;
    font-size: 0.95rem;
}

.table-row:hover {
    background: #f9f9f9;
}

.loading-state {
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 60px;
    color: #999;
    text-align: center;
}

.spinner {
    width: 30px;
    height: 30px;
    border: 3px solid #ddd;
    border-top-color: #0066cc;
    border-radius: 50%;
    animation: spin 1s linear infinite;
    margin-right: 12px;
}

```

```

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

.device-name {
  font-weight: 600;
  color: #0066cc;
}

.heart-rate-cell {
  font-weight: 700;
  color: #e74c3c;
}

.status-badge {
  display: inline-block;
  padding: 4px 8px;
  border-radius: 4px;
  font-size: 0.8rem;
  font-weight: 600;
  text-align: center;
  min-width: 60px;
}

.status-online {
  background: #d4edda;
  color: #155724;
}

.status-offline {
  background: #f8d7da;
  color: #721c24;
}

.no-data-message {
  padding: 40px;
  text-align: center;
  color: #999;
}

.no-data-message-icon {
  font-size: 3rem;
  margin-bottom: 15px;
}

.no-data-message h3 {
  color: #666;
  margin-bottom: 10px;
}

@media (max-width: 1024px) {
  .table-header,
  .table-row {
    grid-template-columns: 100px 80px 80px 80px 80px 80px 80px 100px;
    font-size: 0.85rem;
  }

  .control-label {
    min-width: 80px;
  }
}

```

```

    }
  }

  @media (max-width: 768px) {
    .header-content {
      flex-direction: column;
      gap: 15px;
    }

    .nav-tabs {
      width: 100%;
      justify-content: center;
    }

    .stats-grid {
      grid-template-columns: 1fr;
    }

    .table-header,
    .table-row {
      grid-template-columns: 1fr;
      gap: 5px;
    }

    .controls {
      flex-direction: column;
      align-items: flex-start;
    }
  }
</style>
</head>
<body>
  <div class="header">
    <div class="header-content">
      <div class="logo">
        <span>MICROMAX EMERGENCY RESPONSE SYSTEM</span>
      </div>
      <div class="nav-tabs">
        <button class="nav-tab active">Dashboard</button>
        <button class="nav-tab">Analytics</button>
        <button class="nav-tab">Alerts</button>
        <button class="nav-tab">Settings</button>
      </div>
    </div>
  </div>

  <div class="container">
    <div class="stats-grid">
      <div class="stat-card">
        <div class="stat-icon"><img alt="Bluetooth icon" data-bbox="348 754 368 768"/></div>
        <div class="stat-value" id="connectedCount">0</div>
        <div class="stat-label">Connected Devices</div>
      </div>
      <div class="stat-card" style="border-top-color: #dc3545;">
        <div class="stat-icon"><img alt="Warning icon" data-bbox="348 824 368 838"/></div>
        <div class="stat-value" id="activeAlerts" style="color: #dc3545;">0</div>
        <div class="stat-label">Active Alerts</div>
      </div>
      <div class="stat-card">
        <div class="stat-icon"><img alt="Bar chart icon" data-bbox="348 894 368 908"/></div>

```

```

        <div class="stat-value" id="dataPoints">0</div>
        <div class="stat-label">Data Points Today</div>
    </div>
    <div class="stat-card">
        <div class="stat-icon"><img alt="clock icon" data-bbox="348 141 368 156"/></div>
        <div class="stat-value" id="systemUptime">00:00:00</div>
        <div class="stat-label">System Uptime</div>
    </div>
</div>

<div class="section-title">
    <span class="section-icon"></span>
    Live Health Data Table
</div>

<div class="controls">
    <div class="control-group">
        <label class="control-label">Filter by Device:</label>
        <select id="deviceFilter">
            <option value="">All Devices</option>
        </select>
    </div>
    <div class="control-group">
        <label class="control-label">Refresh Rate:</label>
        <select id="refreshRate">
            <option value="2000">2 seconds</option>
            <option value="5000">5 seconds</option>
            <option value="10000">10 seconds</option>
        </select>
    </div>
</div>

<div class="data-table-container">
    <div class="table-header">
        <div class="table-header-item">DEVICE</div>
        <div class="table-header-item">HEART RATE</div>
        <div class="table-header-item">ACCEL X</div>
        <div class="table-header-item">ACCEL Y</div>
        <div class="table-header-item">ACCEL Z</div>
        <div class="table-header-item">GYRO X</div>
        <div class="table-header-item">GYRO Y</div>
        <div class="table-header-item">GYRO Z</div>
        <div class="table-header-item">TIME</div>
    </div>
    <div class="table-body" id="tableBody">
        <div class="loading-state">
            <div class="spinner"></div>
            <span>Loading device data...</span>
        </div>
    </div>
</div>

<script>
class HealthMonitorPro {
    constructor() {
        this.apiBase = window.location.origin;
        this.picoData = {};
        this.startTime = Date.now();
    }
}

```

```

    this.refreshRate = 2000;
    this.setupControls();
    this.startUpdating();
  }

  setupControls() {
    const refreshSelect = document.getElementById('refreshRate');
    refreshSelect.addEventListener('change', (e) => {
      this.refreshRate = parseInt(e.target.value);
    });

    const deviceFilter = document.getElementById('deviceFilter');
    deviceFilter.addEventListener('change', () => {
      this.updateTableDisplay();
    });
  }

  async fetchAllData() {
    try {
      const response = await fetch(`${this.apiBase}/api/all`);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);

      const data = await response.json();
      this.picoData = data;
      this.updateUI();
      return data;
    } catch (error) {
      console.error('Error fetching data:', error);
      return null;
    }
  }

  async fetchStatus() {
    try {
      const response = await fetch(`${this.apiBase}/api/status`);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);

      const status = await response.json();
      document.getElementById('connectedCount').textContent = status.pico_count || 0;
      document.getElementById('dataPoints').textContent = status.total_readings || 0;

      this.updateDeviceFilter(status.connected_picos || []);
    } catch (error) {
      console.error('Error fetching status:', error);
    }
  }

  updateDeviceFilter(devices) {
    const select = document.getElementById('deviceFilter');
    const currentValue = select.value;

    while (select.options.length > 1) {
      select.remove(1);
    }

    devices.forEach(device => {
      const option = document.createElement('option');
      option.value = device;
      option.textContent = `Device ${device}`;
    });
  }

```

```

        select.appendChild(option);
    });
}

updateSystemUptime() {
    const elapsed = Math.floor((Date.now() - this.startTime) / 1000);
    const hours = String(Math.floor(elapsed / 3600)).padStart(2, '0');
    const minutes = String(Math.floor((elapsed % 3600) / 60)).padStart(2, '0');
    const seconds = String(elapsed % 60).padStart(2, '0');
    document.getElementById('systemUptime').textContent = `${hours}:${minutes}:${seconds}`;
}

updateUI() {
    this.updateTableDisplay();
    this.updateSystemUptime();
}

updateTableDisplay() {
    const tableBody = document.getElementById('tableBody');
    const filterValue = document.getElementById('deviceFilter').value;

    if (Object.keys(this.picoData).length === 0) {
        tableBody.innerHTML = `
            <div class="loading-state">
                <span>No devices connected</span>
            </div>
        `;
        return;
    }

    let html = "";
    for (const [picoId, data] of Object.entries(this.picoData)) {
        // Filter by device if selected
        if (filterValue && picoId !== filterValue) {
            continue;
        }

        const bpm = data.heart_rate || 0;
        const accel = data.accelerometer || {};
        const gyro = data.gyroscope || {};
        const lastUpdated = data.last_updated ? new Date(data.last_updated).toLocaleTimeString() : 'N/A';

        html += `
            <div class="table-row">
                <div class="device-name">Device ${picoId}</div>
                <div class="heart-rate-cell">${bpm > 0 ? bpm + ' BPM' : 'N/A'}</div>
                <div>${(accel.x || 0).toFixed(2)}</div>
                <div>${(accel.y || 0).toFixed(2)}</div>
                <div>${(accel.z || 0).toFixed(2)}</div>
                <div>${(gyro.x || 0).toFixed(2)}</div>
                <div>${(gyro.y || 0).toFixed(2)}</div>
                <div>${(gyro.z || 0).toFixed(2)}</div>
                <div>${lastUpdated}</div>
            </div>
        `;
    }

    if (html === "") {
        tableBody.innerHTML = `
            <div class="no-data-message">

```

```

        <div class="no-data-message-icon">🚫</div>
        <h3>No matching devices</h3>
        <p>Try adjusting your filter</p>
    </div>
    `;
  } else {
    tableBody.innerHTML = html;
  }
}

startUpdating() {
  // Fetch all data with configurable refresh rate
  this.fetchAllData();
  this.fetchStatus();

  setInterval(() => {
    this.fetchAllData();
  }, this.refreshRate);

  // Always update status every 5 seconds
  setInterval(() => {
    this.fetchStatus();
  }, 5000);

  // Update uptime every second
  setInterval(() => {
    this.updateSystemUptime();
  }, 1000);
}

// Initialize when page loads
document.addEventListener('DOMContentLoaded', () => {
  new HealthMonitorPro();
});
</script>
</body>
</html>

```

12.3 Appendix C: Wearable Device Housing

12.3.1 Initial CAD Images

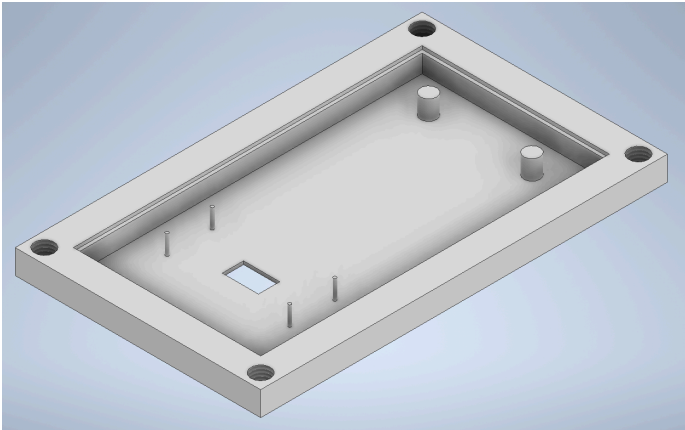


Figure 14: Wearable Device Base Layer

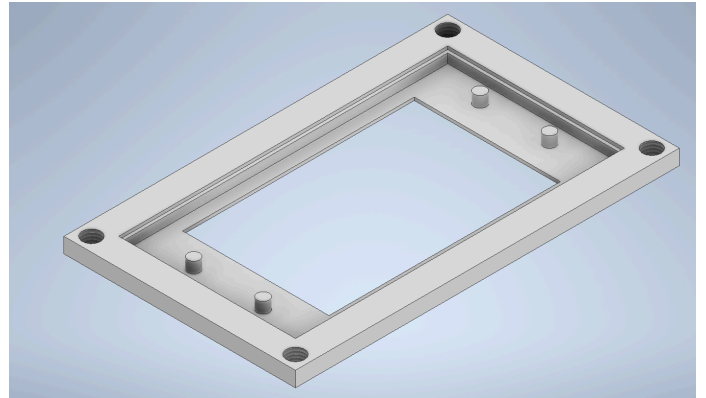


Figure 15: Wearable Device Raspberry Pi Layer

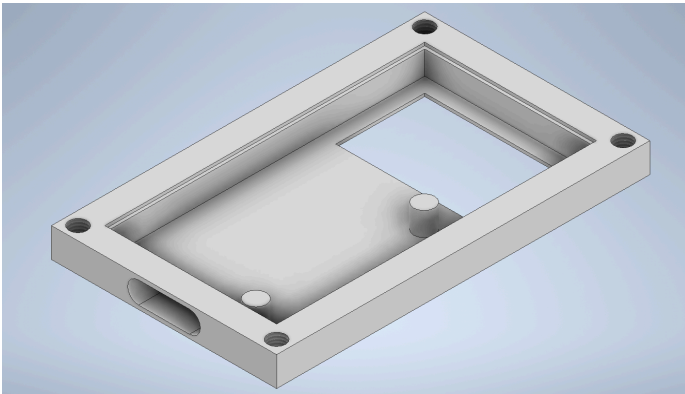


Figure 16: Wearable Device Charger Layer

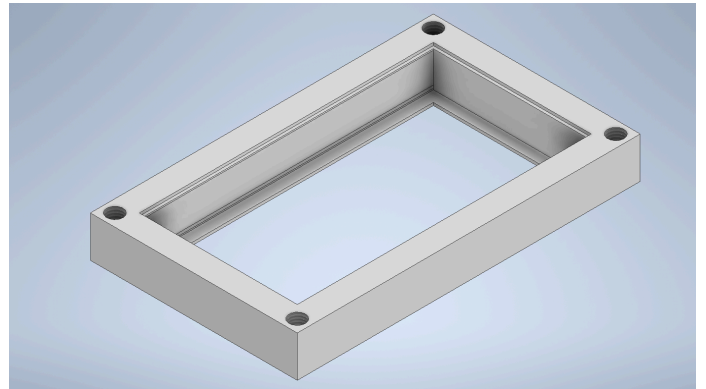


Figure 17: Wearable Device Battery Layer

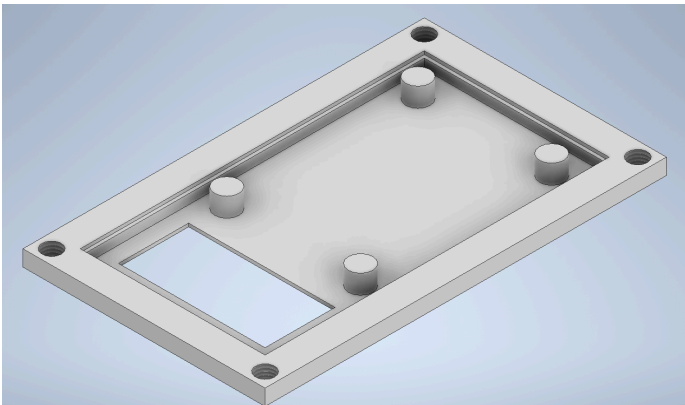


Figure 18: Wearable Device GPS Layer

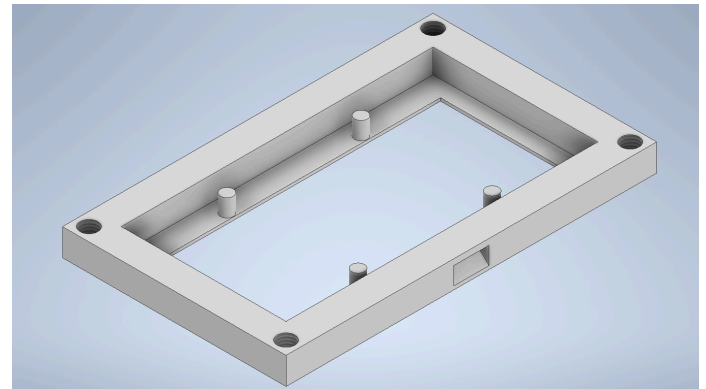


Figure 19: Wearable Device OLED Layer

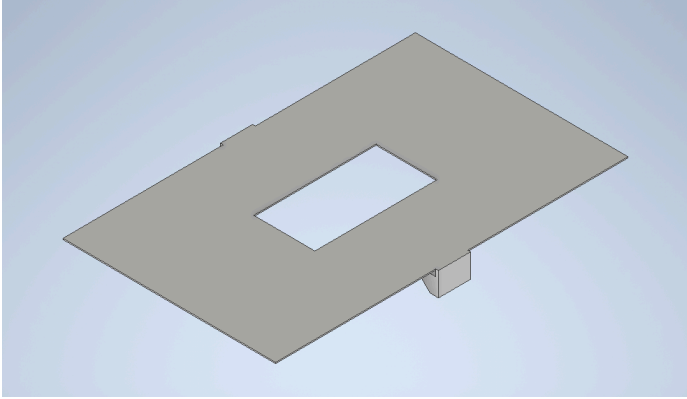


Figure 20: Wearable Device OLED Cover Top View

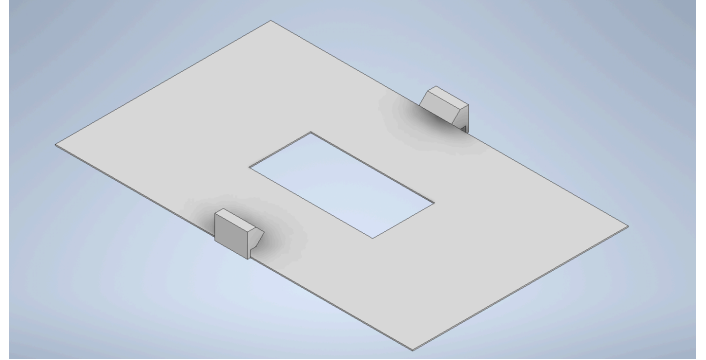


Figure 21: Wearable Device OLED Cover Bottom View

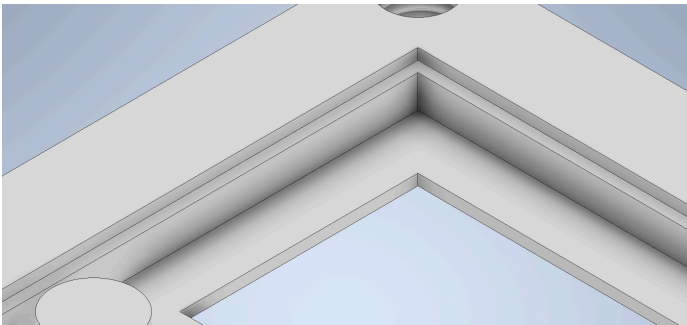


Figure 22: Wearable Inner Rail Close Up

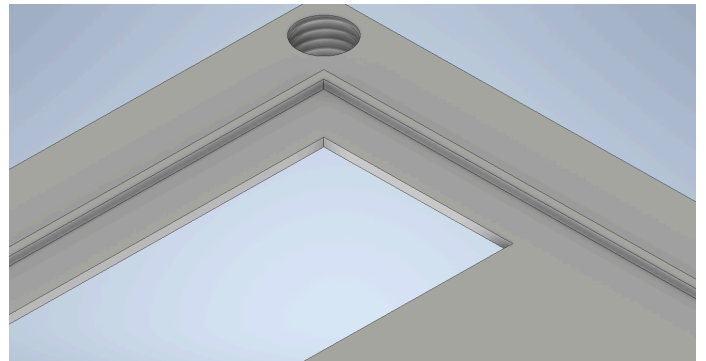


Figure 23: Wearable Outer Rail Close Up

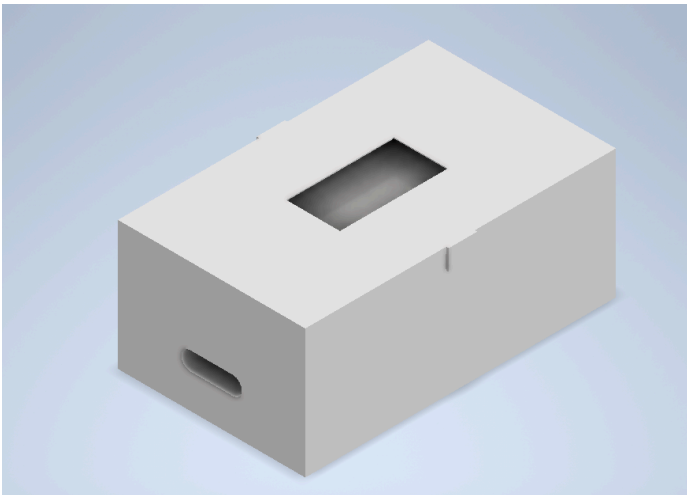


Figure 24: Wearable Device Assembly Top View

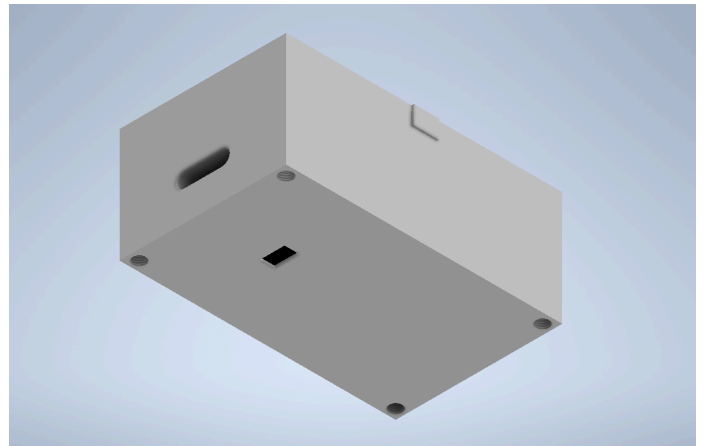


Figure 25: Wearable Device Assembly Bottom View

12.3.2 Final CAD Images

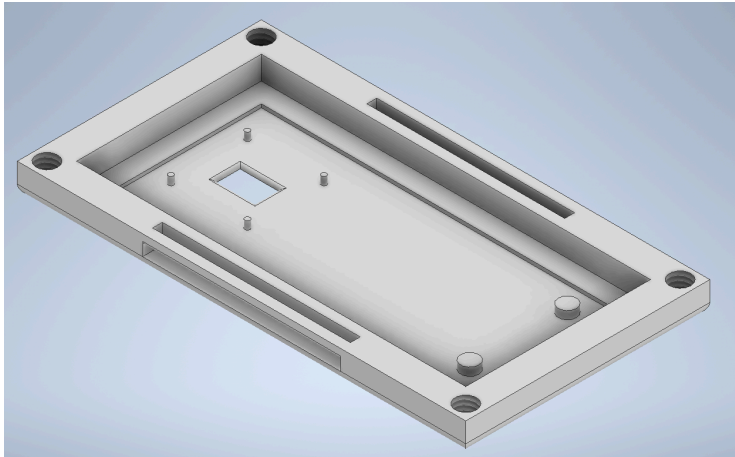


Figure 26: Base Layer Top View

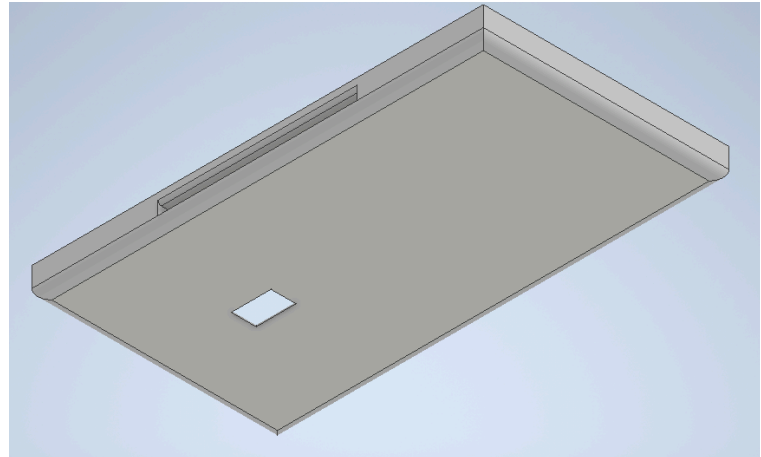


Figure 27: Base Layer Bottom View

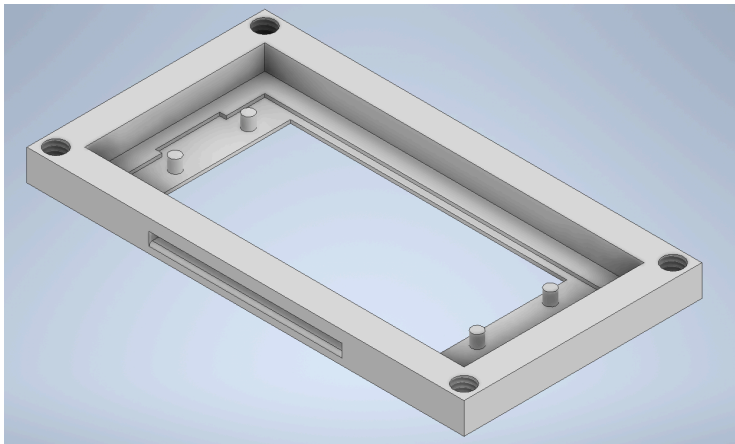


Figure 28: Pico Layer Top View

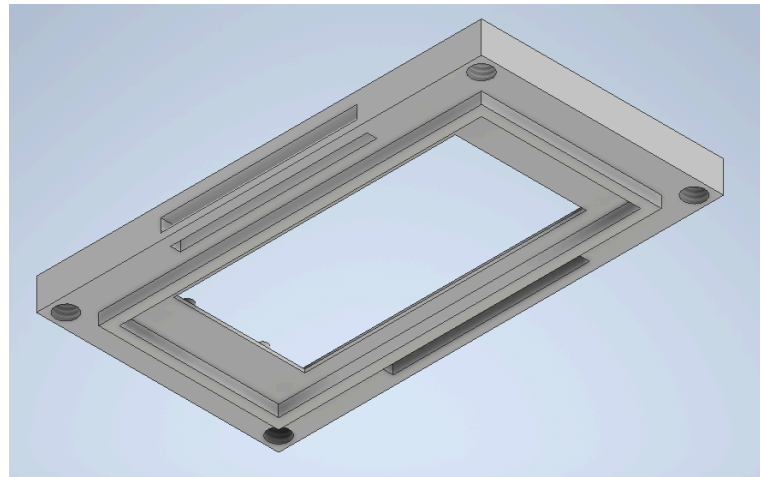


Figure 29: Pico Layer Bottom View

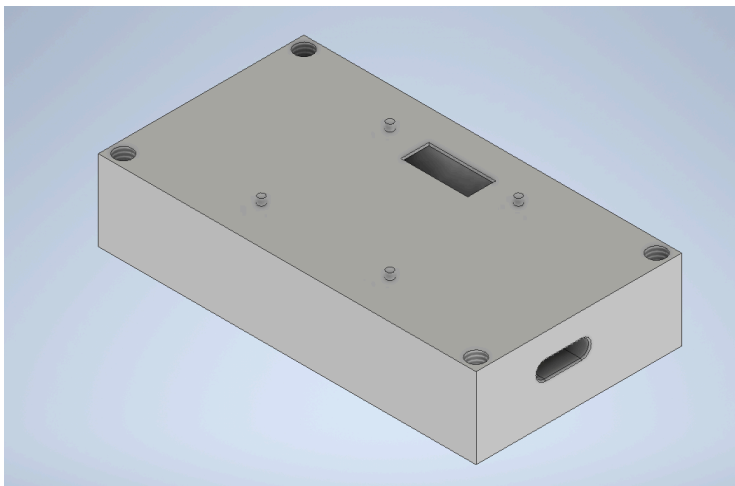


Figure 30: Charger Layer Top View

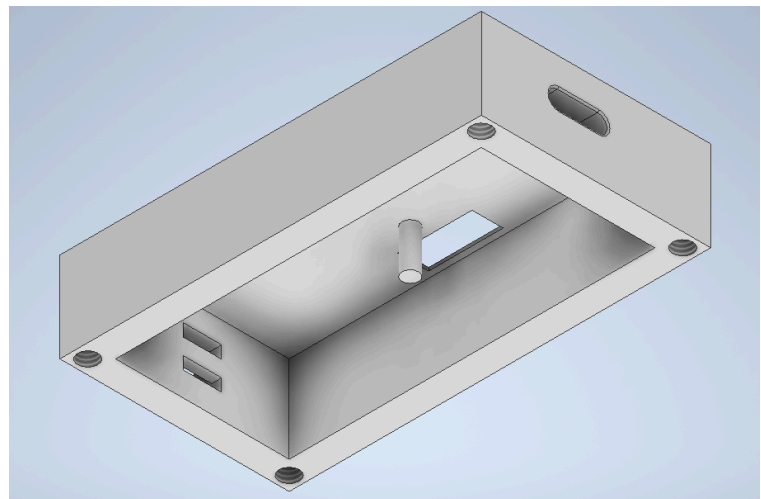


Figure 31: Charger Layer Bottom View

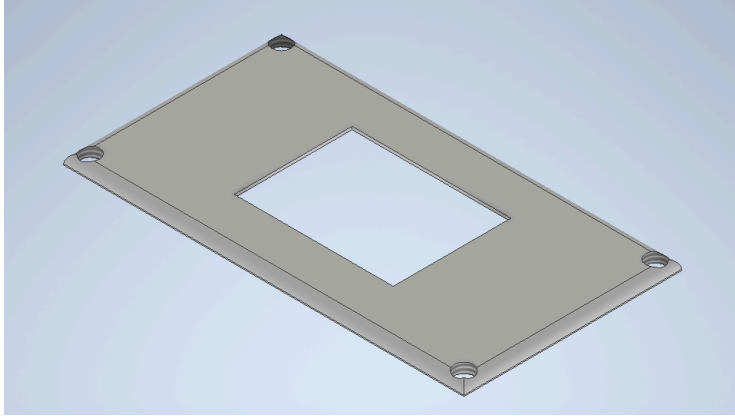


Figure 32: Screen Layer Top View

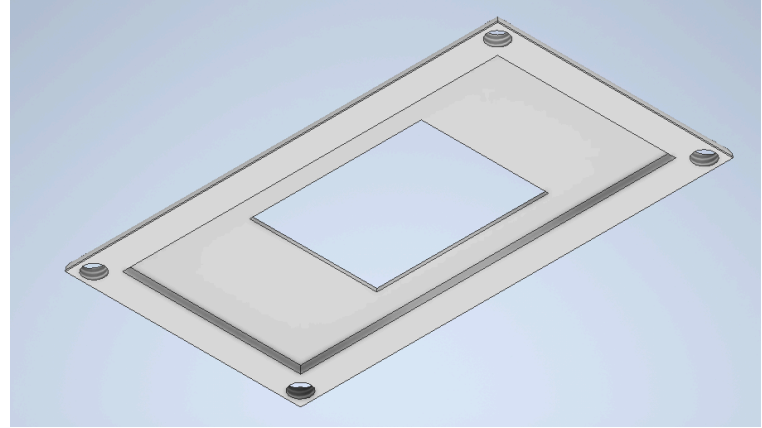


Figure 33: Screen Layer Bottom View

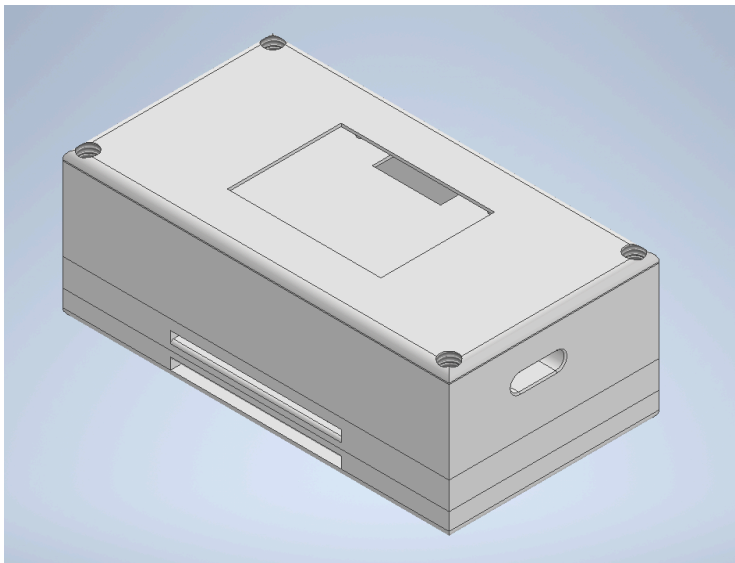


Figure 34: Assembled Device Top View

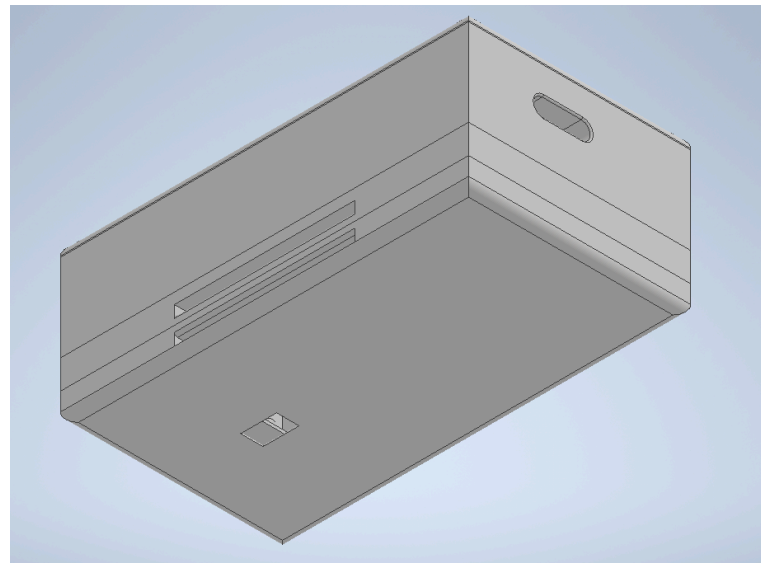


Figure 35: Assembled Device Bottom View

12.3.3 Final Dimensioned Drawing

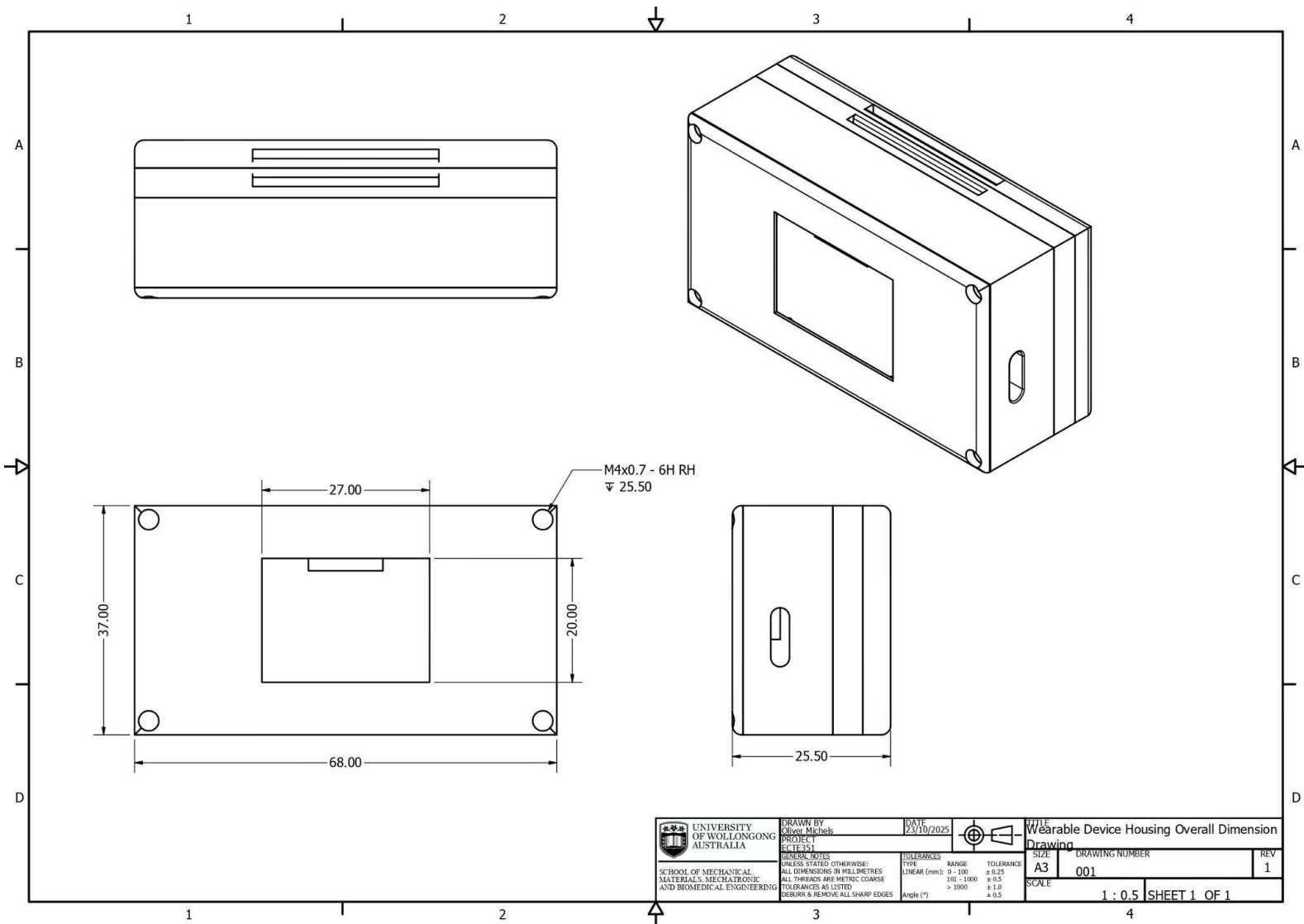


Figure 36: Central Hub Overall Dimension Drawing

12.3.4 Final Physical Images



Figure 37: Disassembled Housing With Corresponding Components Beside

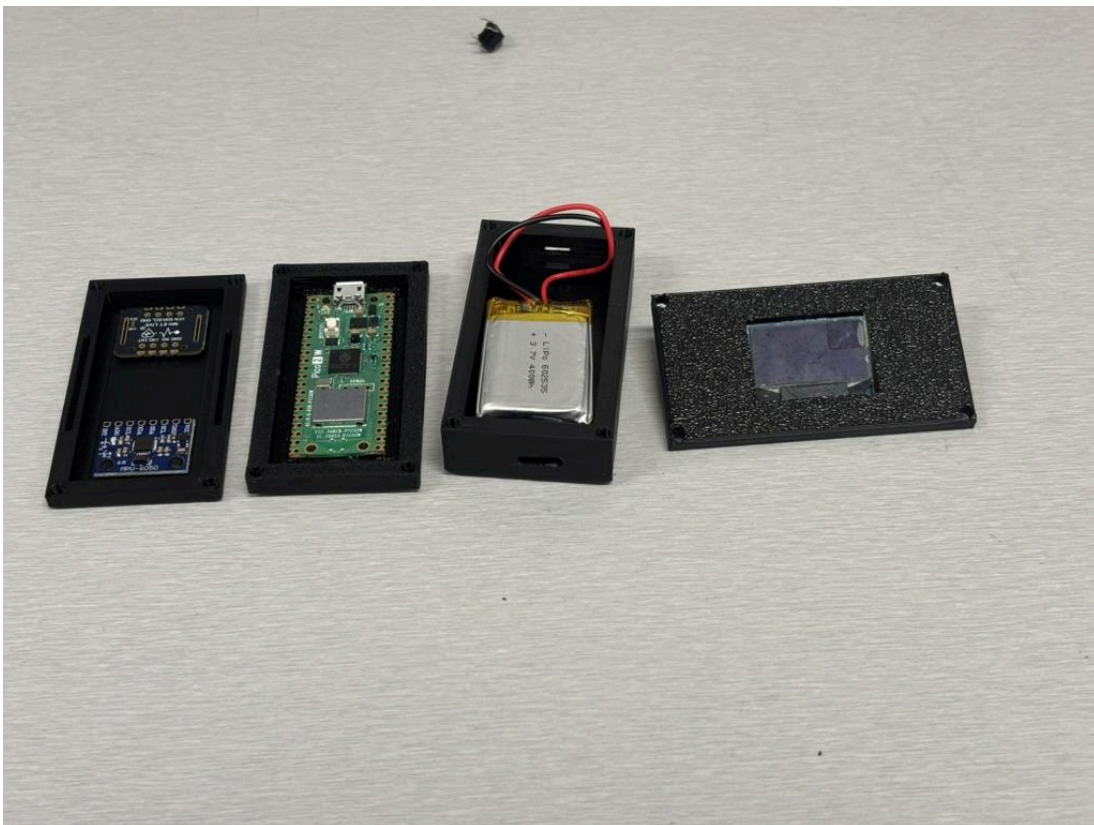


Figure 38: Disassembled Housing With Corresponding Components In Place



Figure 39: Assembled Housing With Components In Place

12.4 Appendix D: Central Hub Housing

12.4.1 Initial CAD Images

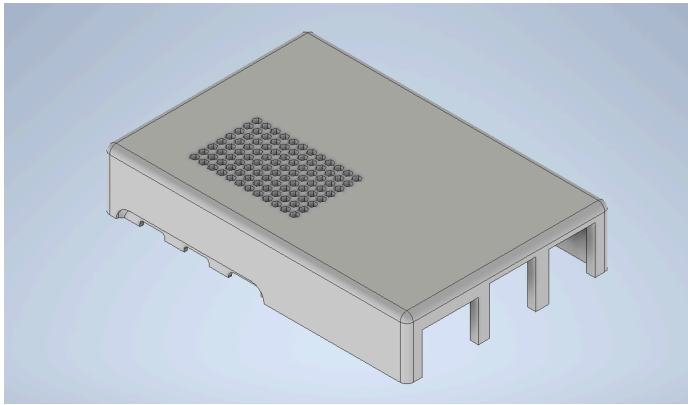


Figure 40: Central Hub Top View

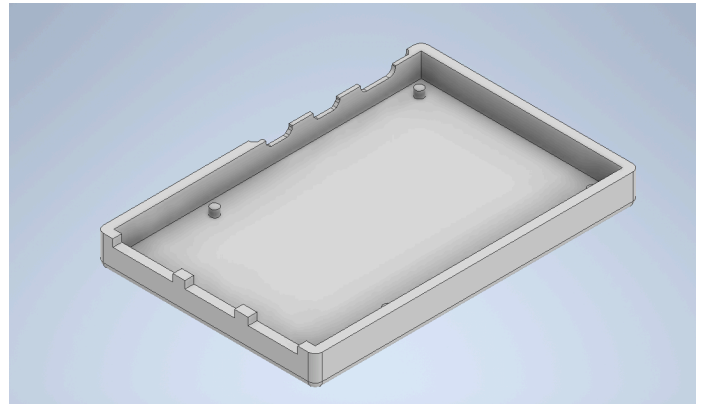


Figure 41: Central Hub Bottom View

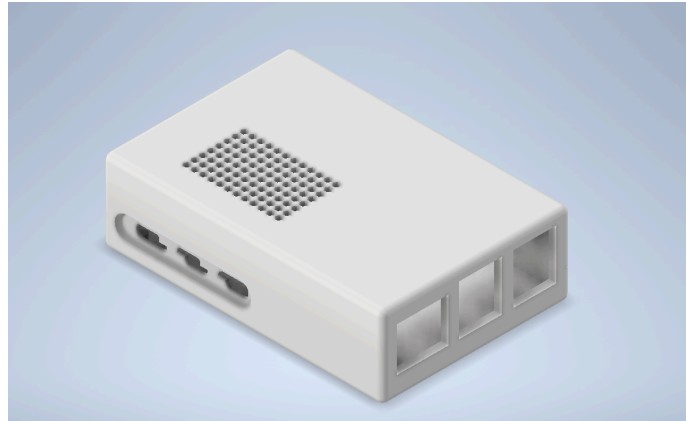


Figure 42: Central Hub Assembly

12.4.2 Final CAD Images

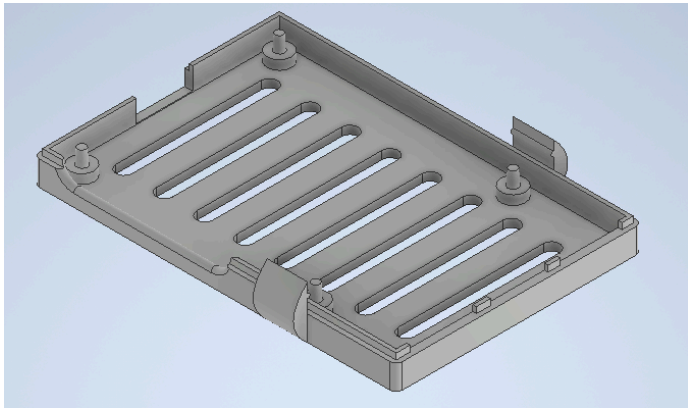


Figure 43: Bottom Layer Top View

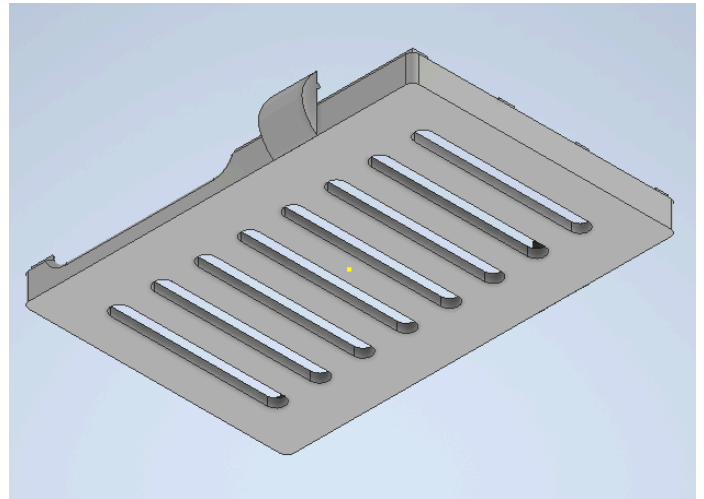


Figure 44: Bottom Layer Bottom View

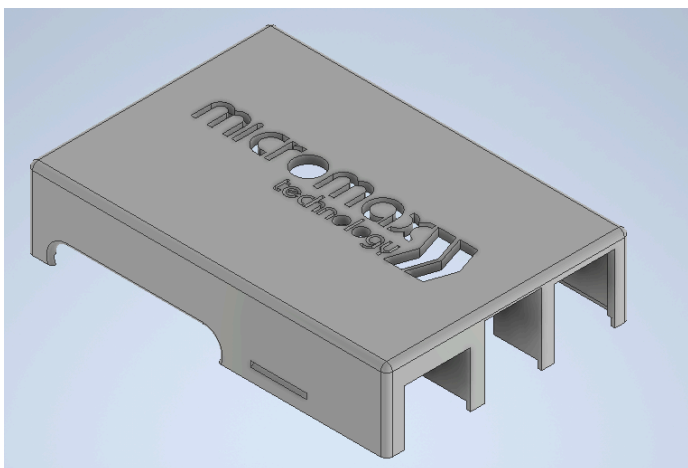


Figure 45: Top Layer Top View

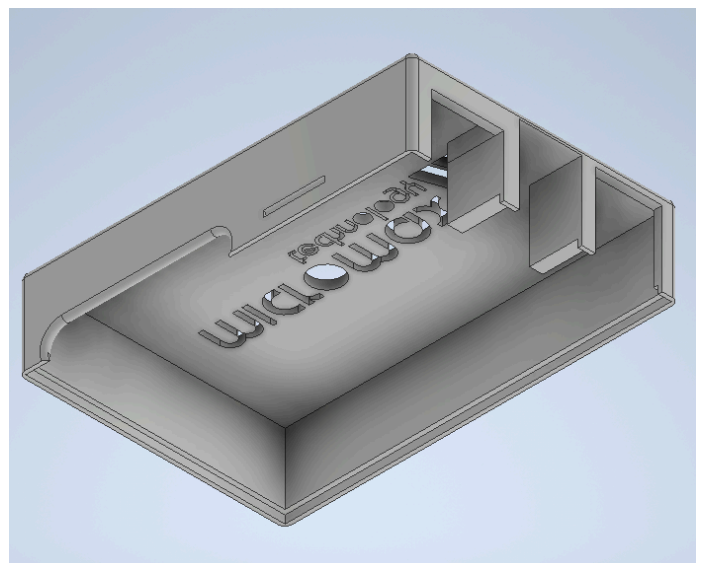


Figure 46: Top Layer Bottom View

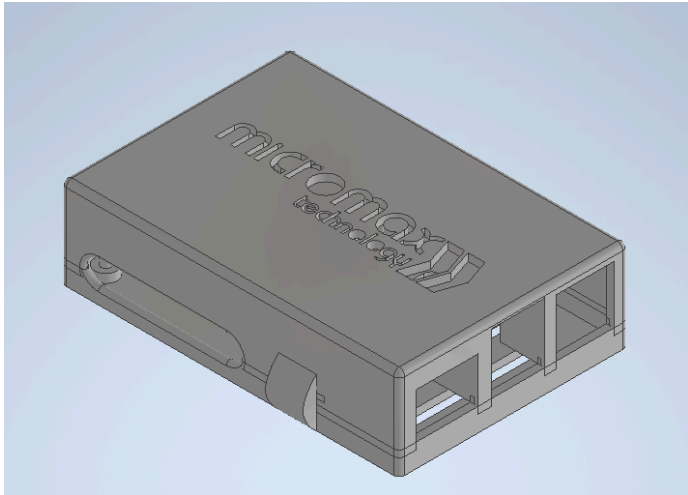


Figure 47: Assembled Device Top View

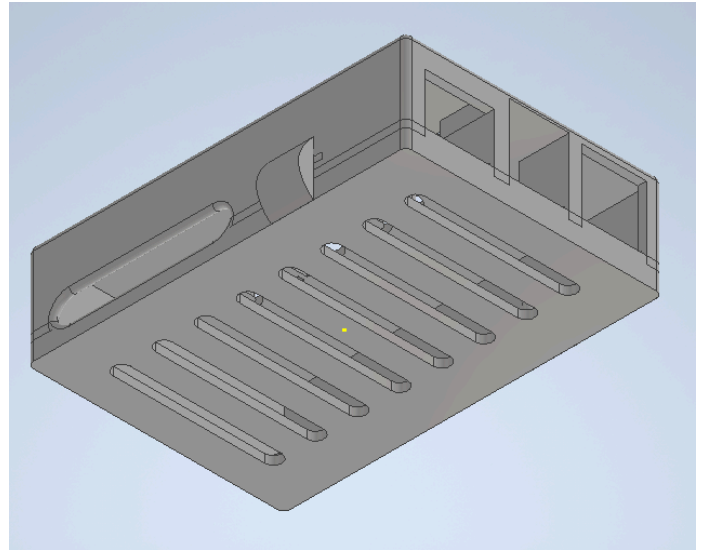


Figure 48: Assembled Device Bottom View

12.4.3 Final Overall Dimension Drawing

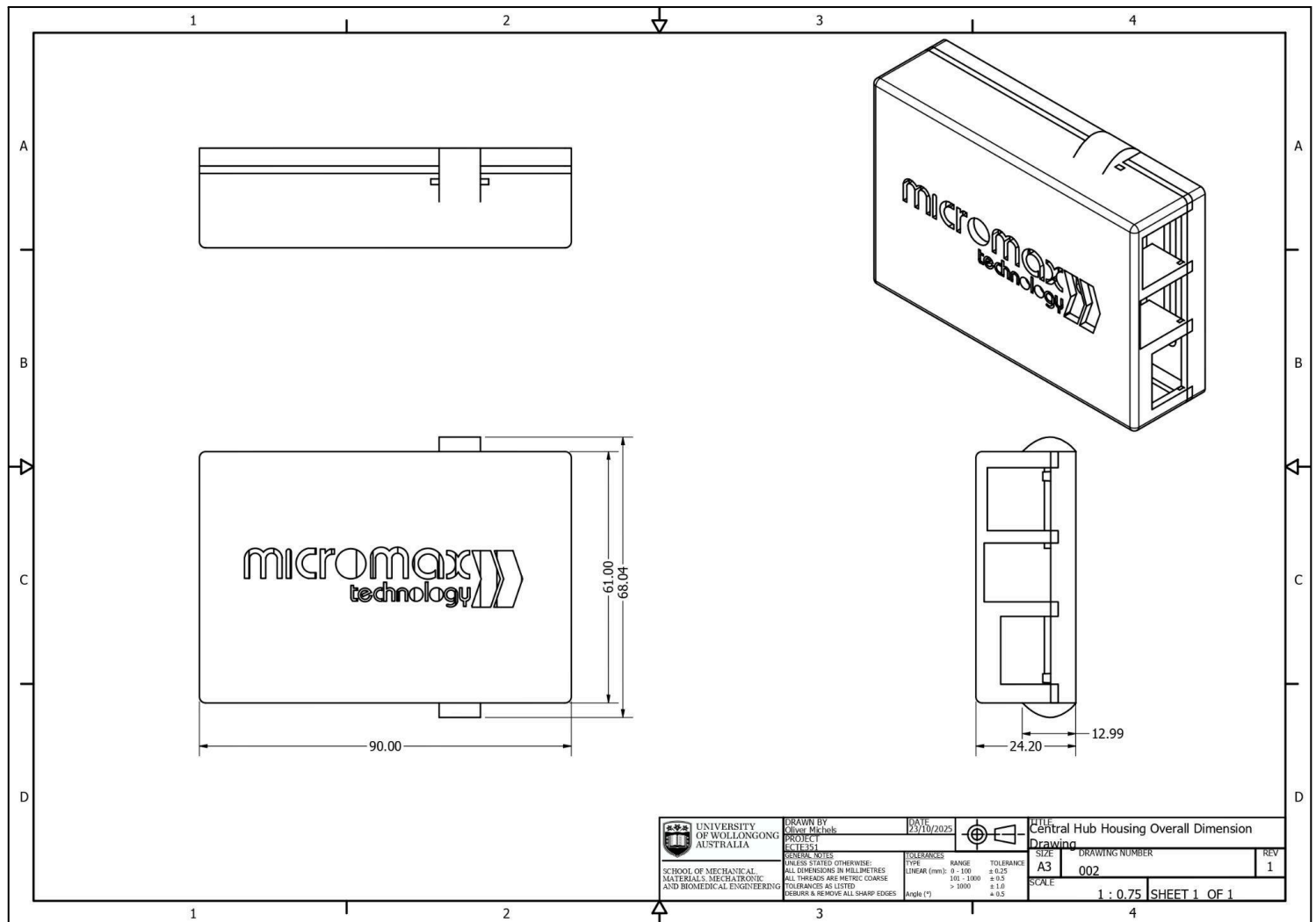


Figure 49: Central Hub Overall Dimension Drawing

12.4.4 Final Physical Images

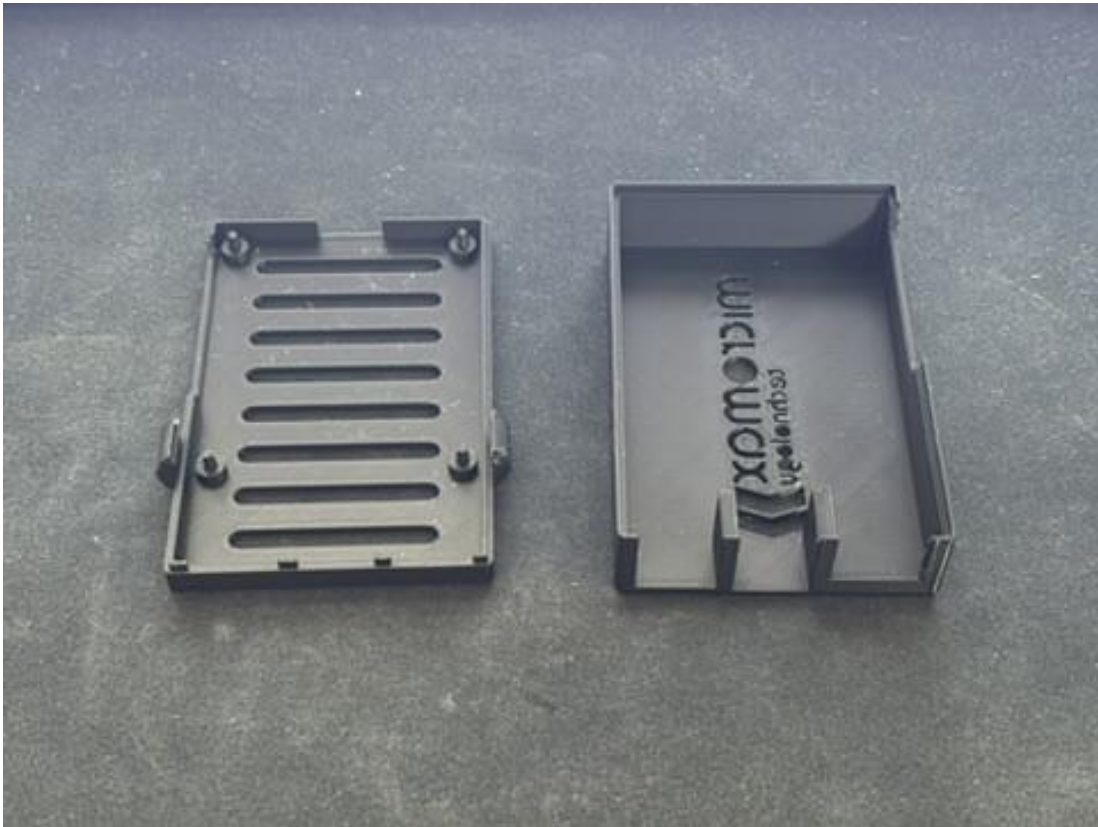


Figure 50: Disassembled Housing



Figure 51: Assembled Housing

12.5 Appendix E: Cash Flow Forecast Calculations

Using the final parts list, with a cost of \$44.25 per wearable device and \$159.27 per central hub. Adding onto this the man hour assembly cost of \$45 per hour, with an expected assembly time of 30 minutes per wearable device (the central hub assembly is to be disregarded in the calculations as it would take <1 min for assembly with only one central hub per order). The order production cost is calculated to be:

$$C = \$159.27 + n(\$44.25) + n(\$22.50) = \$159.27 + n(\$66.75)$$

Where: - C is the order production cost.
 - n is the number of wearable devices required.

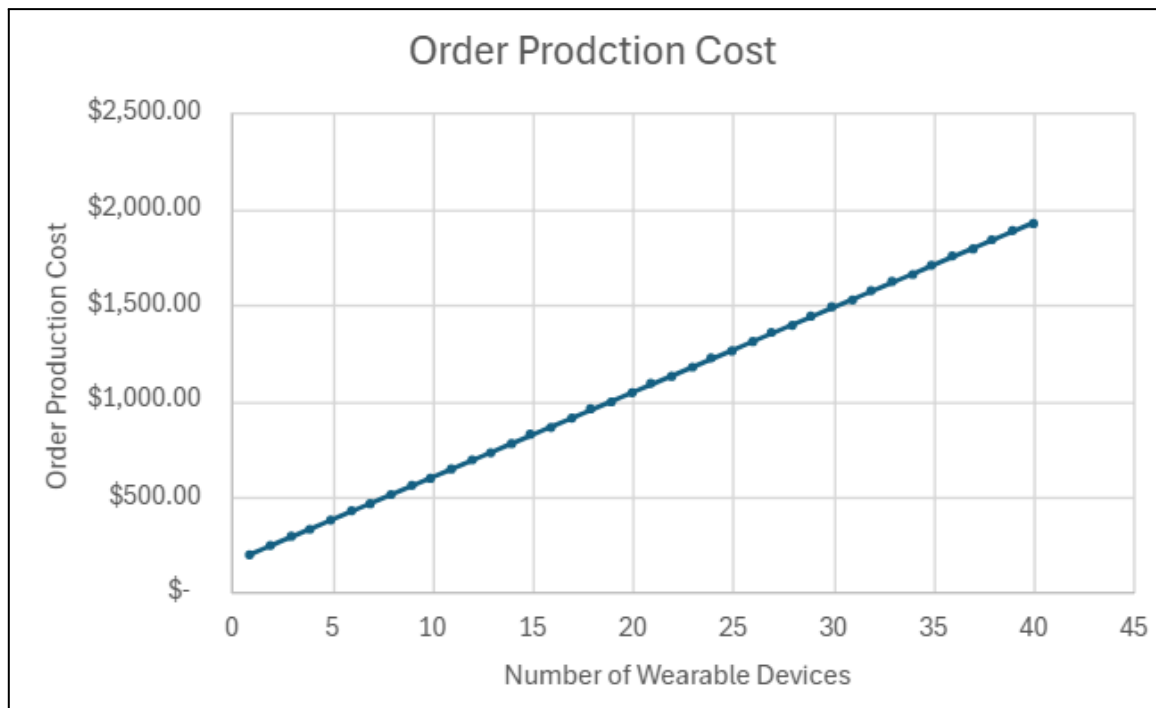


Figure 52: Order Production Cost

This can then be divided by the n , number of wearable devices, to calculate the relative order production cost per wearable device,

$$C_{PWD} = \frac{\$159.27 + n(\$66.75)}{n}$$

Where: - C_{PWD} is the relative order production cost per wearable device.
 - n is the number of wearable devices required.

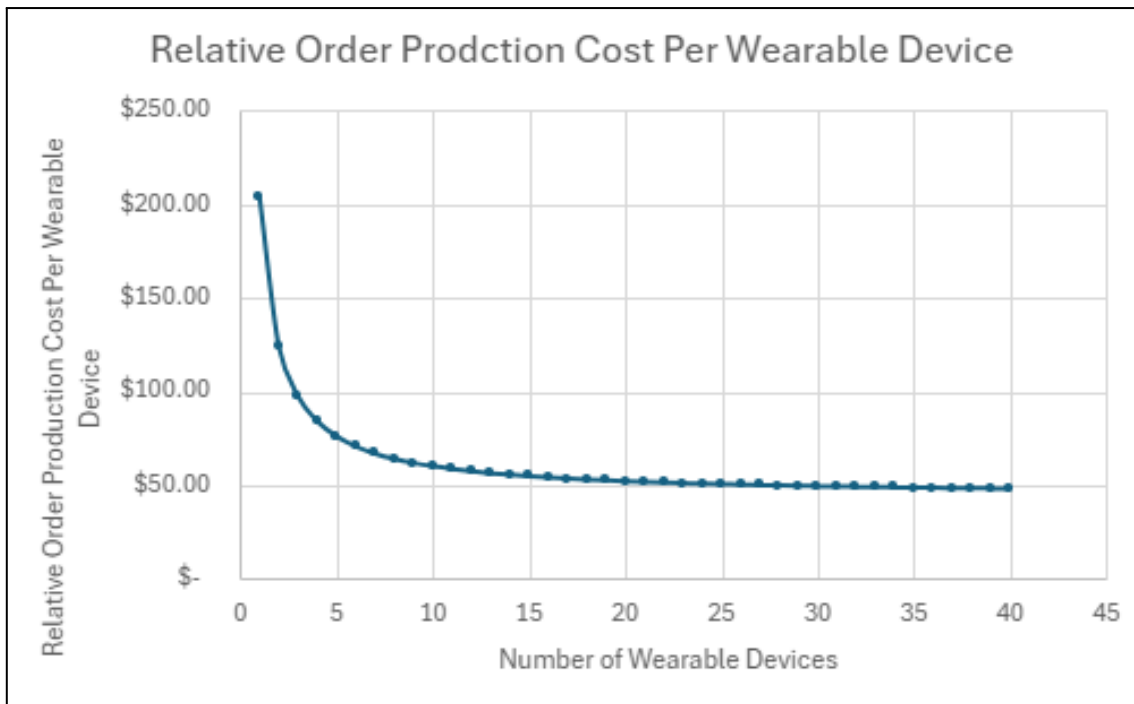


Figure 53: Relative Order Production Cost Per Wearable Device

Estimating the lowest number of devices required in a single order to be 3, with majority of the orders being above this, the pricing of the ERS is to be a flat rate, determinate of the number of wearable devices required, at \$120 per wearable device. This results in a return of investment per wearable device calculated at:

$$ROI_{PWD} = n(\$120) - [\$159.27 + n(\$66.75)]$$

- Where: - ROI_{PWD} is the return on investment per wearable device.
 - n is the number of wearable devices required.

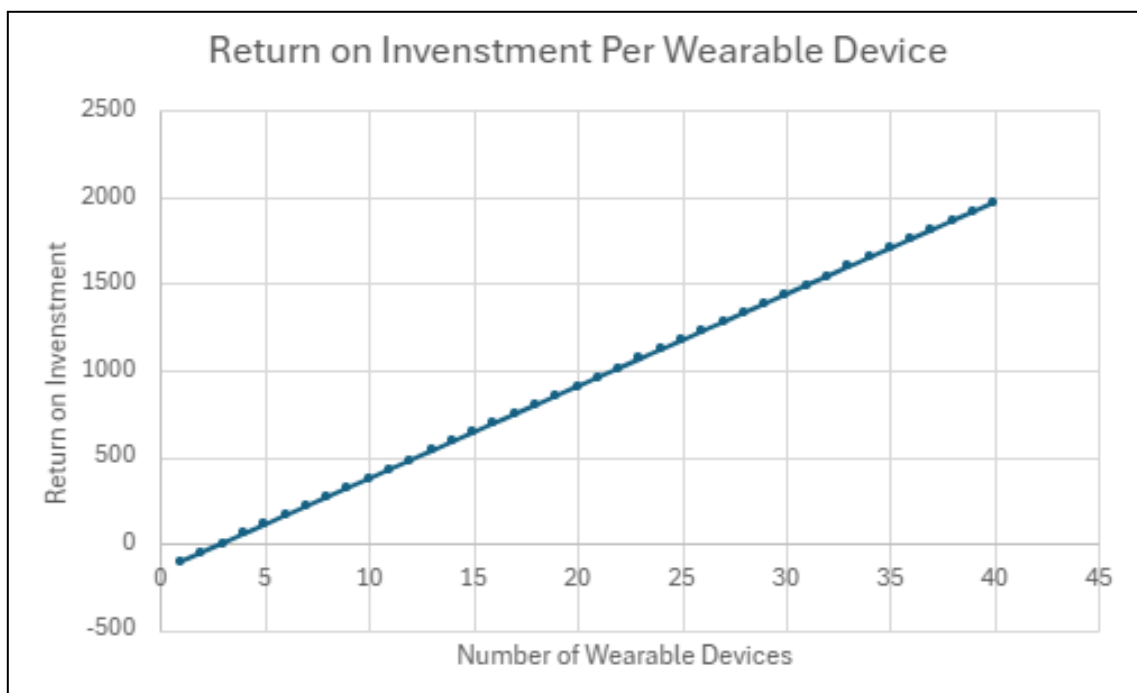


Figure 54: Return on Investment Per Wearable Device

12.6 Appendix F: Gantt Chart

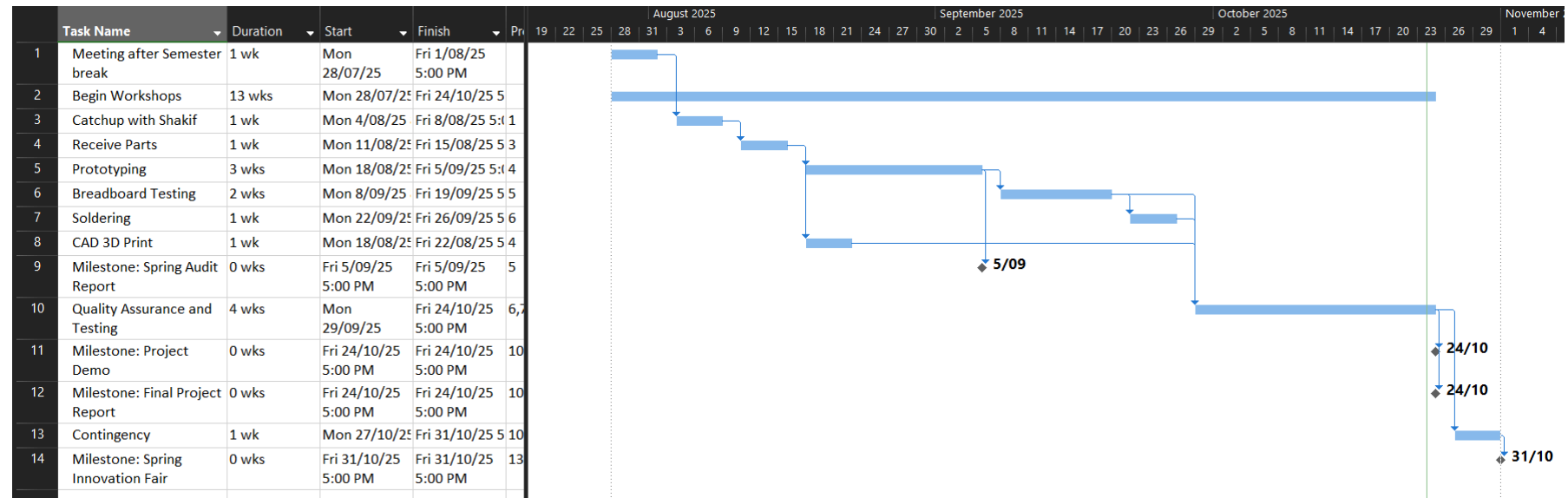


Figure 55: Gantt Chart

12.7 Appendix G: FCS

ECTE351 Fair Contribution Sheet

Team No: 11

Team Name: Group 11

Mark Pool (multiply no. in team by 20): 120

Number of students in Team: 6

FCS Assessment Number: 5&6

Date: 3/6/25

ALL STUDENTS MUST SIGN AGREEMENT

Student Name and Signature:

Zachary Wark Z.W

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20

Student Name and Signature:

Ziyue Li Z.L

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20

Student Name and Signature:

Thomas Speer T.S

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20

Student Name and Signature:

Laila Salem L.S

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20

Student Name and Signature:

Nur Azmina Mohd Rashidi N.R

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20

Student Name and Signature:

Oliver Michels O.M

Assessment Task	Mark
Attendance; Discussion / Team Participation; Participation as Role; Contribution to Deliverable	20